

Reti dei calcolatori 1

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	Il modello OSI	5
1.1	L'incapsulamento del messaggio	5
1.2	Le interfacce ai livelli	6
2	Modelli applicativi per applicazioni di rete	7
2.1	Modello Client-Server	7
2.2	Il modello Peer To Peer	7
2.3	Il modello Publish Subscribe Notify	8
3	Realizzare un web client (HTTP 0.9)	9
3.1	Socket(2)	9
3.2	Socket stream, socket datagram, TCP e UDP	9
3.3	Connect(2)	9
3.4	IPv4	10
3.5	Write(2) e Read(2)	10
3.6	wc_HTTP_0_9.c	11
4	HTTP 1.0	14
4.1	RFC 1945, HTTP 1.0	14
4.2	La grammatica	14
4.3	HTTP Message	16
4.4	wc_HTTP_1_0.c	18
5	HTTP 1.1	21
5.1	Three-way handshake	21
5.2	Connection keep alive	21
5.3	Chunked Transfer Coding e Content-Lenght	22
5.4	wc_HTTP_1_1.c	23
6	Cenni su HTTP 2.0	26
6.1	Framing dello stream e richieste/risposte interleaved	26
7	Cenni su HTTP 3.0	26
7.1	QUIC over UDP	26
8	Realizzare un webserver	27
8.1	Bind(2)	27
8.2	Listen(2)	27
8.3	Accept(2)	27
8.4	ws.c	28
9	Il caching	32
9.1	Il principio del caching	32
9.2	I meccanismi del caching	32
9.3	Confronto tra i meccanismi di caching	35
10	L'autenticazione	37
10.1	Il principio dell'autenticazione	37
10.2	Basic Authentication Scheme	37
10.3	La codifica base64	38
10.4	Basic-cookie	39

11 Common Gateway Interface	41
11.1 URI	41
11.2 Query	42
11.3 Il metodo POST	44
11.4 ws_cgi.c	45
12 Domain Name System	51
12.1 Gli obiettivi del DNS	51
12.2 La struttura dei nomi e la risoluzione di un nome	51
12.3 Gestione dei cambiamenti	52
12.4 Le zone	52
12.5 DNS non-recursive query chain	53
12.6 DNS recursive query	55
12.7 DNS caching	55
13 Il proxy	57
13.1 Il proxy in HTTP	57
13.2 Il proxy in HTTPS	58
13.3 proxy.c	58

1 Il modello OSI

Il modello OSI è un modello standardizzato dall'ISO (*International Standard Organization*). Viene chiamato **reference model**.

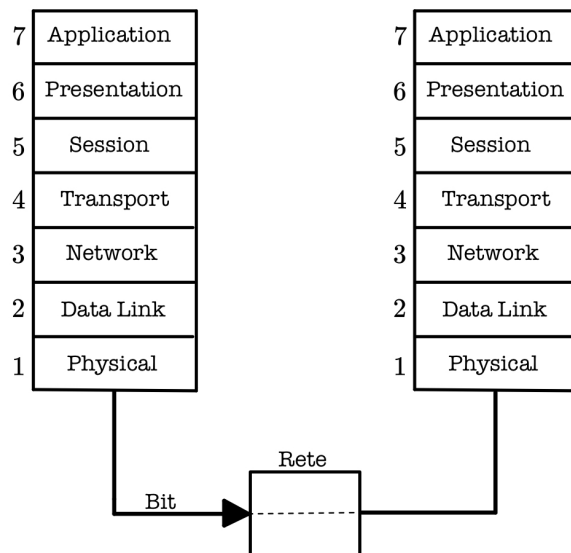


Figura 1: il modello osi.

Si tratta di un modello gerarchico: ogni strato del modello comunica solamente con lo strato superiore e quello inferiore attraverso le rispettive interfacce.

Ogni nodo della rete (computer, smartphone, server, etc.) viene rappresentato in astratto dai 7 livelli.

1.1 L'incapsulamento del messaggio

Il messaggio che deve essere spedito inizia il suo percorso partendo dal livello 7 e man mano prende forma fino ad arrivare al livello 1. Raggiunto il physical layer viene spedito al ricevitore.

Il livello 7 è colui che contribuisce per primo alla creazione, in particolare aggiunge una serie di bit i quali formano il **PDU₇ (Protocol Data Unit)** che di fatto contiene il messaggio che vogliamo inviare. Successivamente il PDU₇ passa al livello sottostante, questo cambia nome e diventa **SDU₆**. Il livello 6 aggiunge all'SDU₆ dei bit: un **header (H6)** di livello 6. L'unione di H6 e SDU₆ forma il **PDU₆**. Dopodiché il PDU₆ passa al livello sottostante, questo cambia nome e diventa **SDU₅**. Il livello 5 aggiunge all'SDU₅ dei bit: un **header (H5)** di livello 5. L'unione tra H5 e SDU₅ forma il **PDU₅**. Si continua in questo modo seguendo la regola $PDU_{i+1} = SDU_i$ fino ad arrivare al livello fisico ($i = 1$). Una volta giunti all'ultimo livello il pacchetto che abbiamo costruito è pronto per essere spedito al mittente.

Nella fase di ricezione, il pacchetto ricevuto subisce il processo opposto: viene spaccettato passando dal livello più basso fino a raggiungere il livello 7.

Il modo in cui vengono impacchettati e spaccettati i pacchetti crea una sorta di comunicazione diretta tra i livelli. Questo è l'intento del modello OSI; in generale è il concetto che sta alla base della rete a pacchetti.

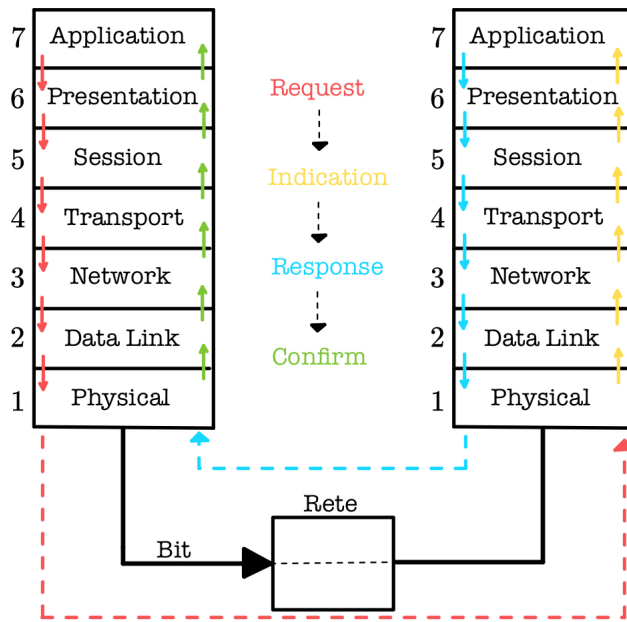


Figura 2: Comunicazione tra i livelli nella creazione e ricezione del messaggio.

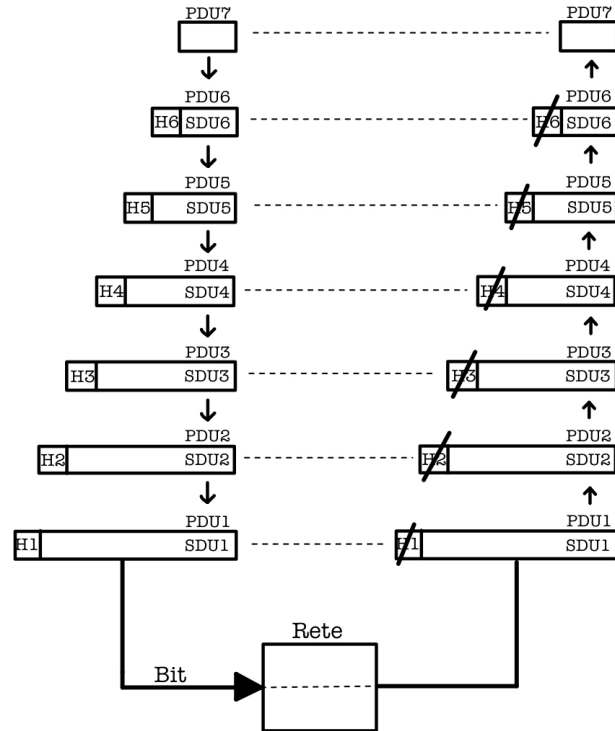


Figura 3: Creazione e ricezione del messaggio.

1.2 Le interfacce ai livelli

Le interfacce ai livelli (chiamate **Service Access Point, SAP**) possono essere driver, I/O, interrupt, signals di un processo, delle funzioni che tornano un valore, etc. Il loro compito è quello di definire il metodo con cui il livello n può comunicare con il livello $n - 1$ e il livello $n + 1$.

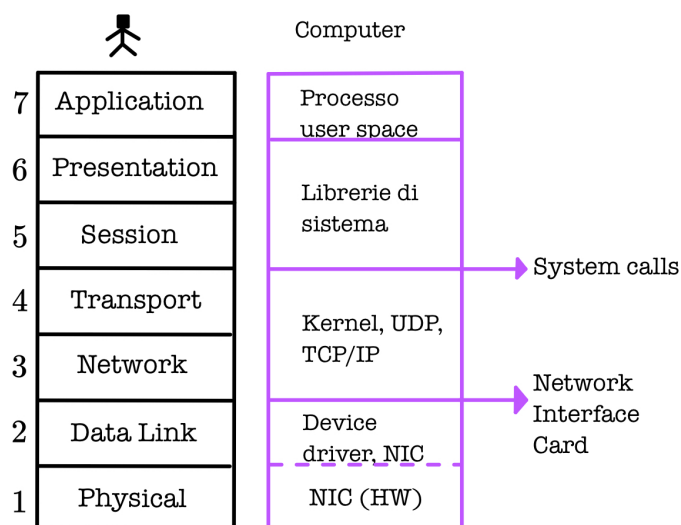


Figura 4: Implementazione dei livelli e interfacce.

2 Modelli applicativi per applicazioni di rete

2.1 Modello Client-Server

Un modello client-server è composto da due programmi che girano (non obbligatoriamente su computer diversi). Uno viene identificato con il nome di **client** e l'altro con **server**; i nomi identificano un ruolo asimmetrico. In particolare: il client fa una richiesta (**request**) al server; il server risponde (**response**) al client.

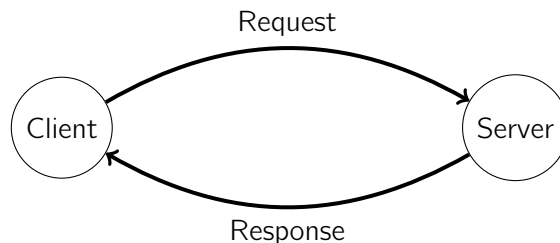


Figura 5: Modello client-server. La request e la response corrispondono a quelle di Figura 2.

La differenza sostanziale tra il ruolo del client e quello del server è che: il client prende l'iniziativa, è colui che scatena l'operazione e sceglie il server a cui inoltrare la richiesta; il server invece riceve e risponde a richieste asincrone (potenzialmente) da più client. Due parametri che caratterizzano il server sono:

- il **throughput** che indica quante richieste al minuto vengono evase;
- la **latenza** che indica il tempo di servizio e response time.

Il limite del modello client-server è che si presuppone che il server risponda sempre dopo una richiesta di un client. Può anche impiegare molto tempo (latenza alta) ma deve rispondere. In certe situazioni potrebbe capitare l'informazione richiesta dal client potrebbe ancora non essere disponibile, potrebbe non esistere ancora. Questi casi non vengono gestiti dal modello client-server, serve un altro modello.

2.2 Il modello Peer To Peer

Il modello peer-to-peer non è un modello diverso rispetto a quello client-server. La differenza rispetto a quello precedente è che ogni nodo della rete ricopre entrambi i ruoli.

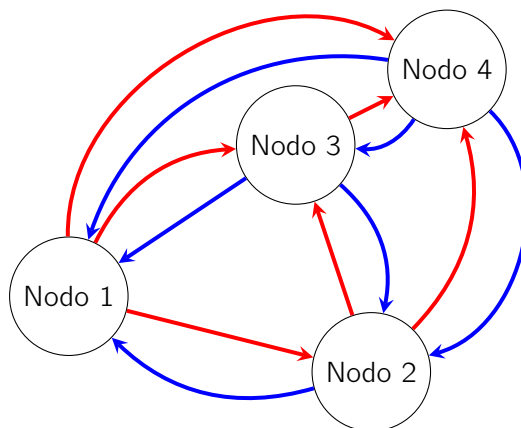


Figura 6: Modello peer-to-peer. Ogni nodo ricopre entrambi i ruoli.

2.3 Il modello Publish Subscribe Notify

Nel modello Publish-Subscribe-Notify: il **subscriber** invia una richiesta; quando il **publisher** pubblica le informazioni, il **provider** notifica il subscriber inviandogli le informazioni che aveva richiesto. Più precisamente:

- **Subscription**: definisce le regole legate al publisher e ai contenuti pubblicati.
- **Publish**: si tratta di un'azione asincrona (il publisher prende l'iniziativa).
- **Notify**: si tratta di un'azione sincrona che segue la publish; è determinata dal match tra publish e subscription.

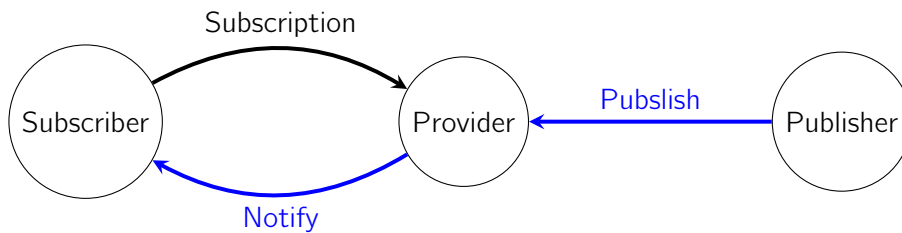


Figura 7: Modello Publish-Subscribe-Notify.

Un esempio di applicazione che sfrutta questo modello è Whatsapp: quando entro in un gruppo mi iscrivo; inizio ad essere notificato quando qualcuno scrive un messaggio. Il provider sono i server di Whatsapp, io che ricevo il messaggio sono il subscriber e la persona che invia il messaggio è il publisher. In generale, non tutte le applicazioni che inviano notifiche utilizzano questo modello.

Differenze tra Publish-Subscribe-Notify e Client-Server La principale differenza è che il modello Client-Server ha come paradigma la chiamata a funzione, mentre il Publish-Subscribe-Notify ha come paradigma l'interrupt handling.

Questi due modelli rappresentano i capisaldi delle reti, sono i mattoni con cui vengono costruite le applicazioni di rete. Partendo da questi si possono costruire (componendoli) modelli molto più complessi.

3 Realizzare un web client (HTTP 0.9)

L'obiettivo che ci poniamo è quello di realizzare un web client che si colleghi al server di google.

3.1 Socket(2)

Il primo passo per realizzare un web client è quello di comprendere come funziona un **socket**. Per farlo analizziamo la system call '`int socket(int domain, int protocol, int type)`' che si trova nella seconda sezione (quella dedicata alle system call) del manuale di socket. Attraverso questa system call possiamo creare un *endpoint*, ovvero un collegamento che permette la comunicazione tra due dispositivi. Analizziamo i parametri della funzione:

- `int domain`: siamo interessati alla famiglia di protocolli Internet, in particolare a `AF_INET` (internet protocol IPv4). "AF" sta per *Address Family*.
- `int protocol`: nei protocolli di Internet, le due principali tipologie di servizi usati sono `SOCK_STREAM` e `SOCK_DGRAM`. La scelta tra i due dipende dal tipo di applicazione che si vuole realizzare. Nella sottosezione successiva analizziamo le differenze tra i due.
- `int type`: nella maggior parte dei casi viene settato a zero. Si tratta di un parametro usato per identificare sotto-protocolli del protocollo specificato nel parametro precedente. Nel nostro caso setteremo il valore a zero.

3.2 Socket stream, socket datagram, TCP e UDP

La differenza tra i servizi `SOCK_STREAM` e `SOCK_DGRAM` consiste nel modo in cui i dati vengono trasmessi. Il primo è orientato allo stream, il secondo ai datagrammi.

Servizio orientato allo Stream Il servizio orientato allo stream funziona in maniera simile a un trasferimento di file. In pratica viene stabilito un **canale di comunicazione bidirezionale** tra due endpoint; i dati inviati arrivano nell'ordine in cui sono stati inviati, confini distinti tra i messaggi (flusso continuo di dati). Il servizio stream viene utilizzato in tutte quelle applicazioni in cui è fondamentale che i dati arrivino completi e nell'ordine corretto (come nel caso di un trasferimento di file).

L'affidabilità del servizio è delegata al protocollo che lavora al livello sottostante (tipicamente **TCP**), questo si occupa infatti dei riconoscimenti, di ritrasmissioni di pacchetti persi e controllo della congestione.

Servizio orientato ai datagrammi Il servizio orientato ai datagrammi è più simile ad inviare una serie di lettere postali: ciascun messaggio (o datagramma) è indipendente dagli altri, non c'è garanzia nell'ordine di arrivo e non c'è nemmeno la certezza che arrivino affatto. Questa tipologia di servizio risulta molto utile per quei tipi di applicazioni (es: chiamate real time, sistemi di comando remoto) in cui conta di più la velocità che l'affidabilità completa.

3.3 Connect(2)

Ora che abbiamo capito come creare un socket, dobbiamo capire come raggiungere un server. Per farlo si sfrutta la system call '`int connect(int sockfd, const struct sockaddr* addr, socklen_t addrlen)`' le cui informazioni si trovano nella sezione 2 del manuale di connect. Analizziamo i parametri:

- `int sockfd`: si tratta del *file descriptor* tornato dalla funzione `socket()`.

- `const struct sockaddr* addr`: indirizzo del server al quale vogliamo connetterci. Nel nostro caso vogliamo connetterci al server di google.
- `socklen_t addrlen`: `sizeof` di `addr`.

Nella sezione 7 (sezione riguardante le informazioni) del manuale di IP troviamo le informazioni che ci servono sulla `'struct sockaddr_in'`. La `struct sockaddr` e la `struct sockaddr_in` hanno entrambe la prima variabile di istanza che identifica la famiglia (`AF_INET` nel nostro caso). In questo modo nella funzione `connect()` possiamo passare un puntatore di tipo `struct sockaddr` (tramite un cast); nella funzione ci sarà un'istruzione che controllerà la famiglia di appartenenza del socket e gestirà caso per caso. Abbiamo descritto come si realizza il polimorfismo in C.

3.4 IPv4

Il protocollo di rete IPv4 prevede che gli indirizzi di rete siano di 4 byte, ad esempio 147.162.2.100. Attraverso l'indirizzo IP viene identificata univocamente un'interfaccia di rete connessa ad internet. Il livello 3 si occupa di recapitare il messaggio al destinatario specificato. L'indirizzo IP del server con cui vogliamo comunicare non ci basta, ci serve sapere anche il **port** per identificare il socket, ovvero l'endpoint del server. Il port è un numero a 16 bit unico all'interno del server e specifico per un tipo di programma server. Vedremo più dettagli quando studieremo la comunicazione lato server.

La domanda ora è: come faccio io, client, sapere il numero del port del server? I programmi server utilizzano dei numeri di port *well known* che identificano il tipo di protocollo utilizzato. In `/etc/services` è possibile visualizzare tutti i numeri standard associati ai protocolli. Uno dei più utilizzati (e quello che utilizziamo nel nostro caso) è il port 80, quello di http.

Fissato il port, ci serve l'indirizzo del server. Per ricavarlo possiamo utilizzare il comando `nslookup <address>`. Siamo interessati al server di google, quindi utilizziamo `nslookup www.google.com`. Il comando restituisce sia l'indirizzo IPv4 che quello IPv6 (da 128 bit).

3.5 Write(2) e Read(2)

Ora che sappiamo come aprire uno stream e come collegarci ad un server, dobbiamo capire come avviene la comunicazione. Il socket che abbiamo aperto è *full duplex*, ovvero permette la scrittura e lettura simultanea. In pratica, noi client, vediamo il server come un file in cui scrivere e viceversa il server vede noi come un file in cui scrivere.

Per poter scrivere e leggere si sfruttano le stesse system call che si utilizzano per scrivere e leggere file. Nel modello client-server è compito del client prendere l'iniziativa, dunque, dopo esserci connessi, dobbiamo inviare una *request* al server. Per farlo usiamo la system call `'ssize_t write(int fd, const void buf[.count], size_t count)'`, dove:

- `int fd`: file descriptor del file in cui vogliamo scrivere (il file descriptor fornito dalla chiamata `socket` nel nostro caso);
- `const void buf[.count]`: buffer contenente il messaggio da scrivere;
- `size_t count`: numero di byte da scrivere nel file a partire dall'inizio del buffer.

Nel nostro caso il messaggio che andremo a scrivere è `"GET <path_file.html>"` e il server ci risponderà inviandoci il contenuto del file specificato. Si tratta del protocollo HTTP 0.9, il primo protocollo HTTP inventato alla fine degli anni ottanta da Tim Berners-Lee. Per poter leggere il contenuto del file utilizziamo la system call `'ssize_t read(int fd, void buf[.count], size_t count)'`, dove:

- `int` `fd`: file descriptor del file in cui vogliamo andare a leggere (nel nostro caso il file descriptor del socket che abbiamo aperto);
- `void` `buf[.count]`: buffer in cui salvare i byte letti;
- `size_t` `count`: numero di byte (massimo) da leggere (può leggerne anche meno di quelli specificati).

3.6 wc_HTTP_0_9.c

```
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <netinet/in.h>
4  #include <unistd.h>
5
6  int main() {
7  // ### Creazione socket ###
8      int s;
9      s = socket(AF_INET, SOCK_STREAM, 0);
10
11     /* La funzione socket ritorna un file descriptor. Stampando il valore di 's'
12      ci viene restituito il valore 3. Si tratta di un indice di una tabella
13      memorizzata in kernel space in cui si trovano i vari stream aperti dal
14      processo.
15      In particolare:
16          - Il valore 0 e' associato allo standard output.
17          - Il valore 1 e' associato allo standard input.
18          - Il valore 2 e' associato allo standard error.
19      Se aprissimo un file prima di lanciare la chiamata a 'socket', allora
20      il file avrebbe 3 come valore associato e 's' avrebbe 4. */
21
22     //printf("s = %d\n", s);
23
24     /* Se s = -1 allora significa che l'end point non e' stato creato perche'
25      c'e' stato un error. In tal caso la variabile globale 'errno' (error
26      number) viene setta ad uno specifico valore. */
27
28     //s = socket(14141, SOCK_STREAM, 0); // Per generare un errore.
29     //if (s == -1) {
30     //    printf("errno = %d\n", errno); // errno = EAFNOSUPPORT = 97.
31     //    perror("Socket fallita");
32     //    return 1;
33     //}
34
35 // ### Connect ###
36 struct sockaddr_in server;
37 server.sin_family = AF_INET;
38
39 /* Da little endian (mio pc) a big endian (network order).
40    Il network order e' il big endian (scelta arbitraria).
41    In questo modo tutti i tipi di periferiche (big o little endian)
42    possono comunicare. */
43 server.sin_port = htons(80);
44
45 /* L'indirizzo IPv4 e' composto da 4 byte.
```

```

46     Per poter scrivere l'indirizzo come campo di 'server', possiamo
47     sfruttare un puntatore a 'char' e scrivere i byte uno per volta.
48     Un'altro modo e' quello di trasformare l'indirizzo in binario
49     poi di nuovo in decimale e poi usare 'htonl' per passare a big
50     endian. */
51 unsigned char* p = (unsigned char*) &server.sin_addr.s_addr;
52
53 /* Scrivento l'indirizzo in questo modo e' gia' in notazione big endian. */
54 p[0] = 216; p[1] = 58; p[2] = 209; p[3] = 36;
55
56 /* Metodo alternativo: e' meno leggibile e richiede piu' passaggi. */
57 //server.sin_addr.s_addr = htonl(3627733284);
58
59 /* * Far fallire la connect *
60     Per fare fallire la connect basta inserire un indirizzo casuale.
61     Si nota che il programma rimane bloccato in attesa: sta aspettando
62     che arrivi la Confirm da parte del server. Dato che l'indirizzo
63     specificato non esiste, il programma rimane in attesa per un certo tempo,
64     dopodiche' 'connect' termina con l'errore 'Connection
65     timed out'. */
66 //p[0] = 16; p[1] = 58; p[2] = 209; p[3] = 36;
67
68 /* Un altro motivo per il quale la connect puo' fallire e' se la connessione
69     con il server viene rifiutata (errore 'Connection refused'). */
70 //p[0] = 127; p[1] = 0; p[2] = 0; p[3] = 1; // Mi collego a me stesso.
71 //server.sin_port = htons(77); // Non c'e' un socket collegato a questa
72     // porta.
73
74 /* Posso verificare se la connessione e' andata a buon fine con il
75     comando 'netstat -nt | grep "<indirizzo>"'. */
76 int tmp;
77 tmp = connect(s, (struct sockaddr*) &server, sizeof(struct sockaddr_in));
78 if (tmp == -1) {
79     perror("Connect fails");
80     return 1;
81 }
82
83 // ### Write ###
84 /* Nel modello client-server e' il client a prendere l'iniziativa.
85     Inviatio un request al server utilizzando il protocollo HTTP 0.9
86     (si tratta di una Simple-Request).
87     Chiediamo il contenuto della radice. Il '\r\n' sta per "carriage return"
88     e "new line" (specificato dal protocollo come CRLF). */
89 write(s, "GET /\r\n", 7);
90
91 // ### Read ###
92 /* Creo un buffer di 'dim' byte in cui salvare il contenuto della pagina.
93     Poi leggerlo' (al massimo) fino a dim - 1 byte in questo modo non rischio
94     di scrivere il terminatore di stringa in 'response[100000]'. */
95 const int dim = 100000;
96 char response[dim];
97
98 /* * Effettuare una singola lettura *
99     Nota che effettuando una singola lettura non ci viene restituita
100     l'intera pagina. Stiamo usando un 'SOCK_STREAM' e quindi riceviamo
101     un flusso di byte, non dei messaggi. Effettuando una singola lettura

```

```
102     di fatto si vanno a leggere i byte che fino ad ora sono stati
103     inviati dal server e, siccome la CPU e' molto piu' veloce del pacchetto
104     dati che viaggia attraverso la rete, non facciamo in tempo a ricevere
105     l'intera pagina in questo modo. */
106     //tmp = read(s, response, dim - 1);
107     //response[tmp] = 0; // Terminatore di stringa.
108
109     /* Una soluzione sarebbe quella di inserire un'attesa: in questo modo
110     si ottiene l'intera pagina web in quanto il server ha avuto tempo
111     di inviare tutti i byte. Soluzione poco efficiente. */
112     //sleep(2);
113     //tmp = read(s, response, dim - 1);
114     //response[tmp] = 0; // Terminatore di stringa.
115
116     /* * Ricevere l'intera pagina web *
117     Per ricevere l'intera pagina senza dover inserire un'attesa
118     ci basta effettuare molteplici chiamate a 'read'. */
119     int i = 0;
120     for (i = 0; (tmp = read(s, response + i, (dim - 1) - i)); i += tmp) {
121         /* Stampa byte ricevuti per ogni chiamata di read.
122         Vengono inviati a multipli di 1400 byte: si tratta delle dimensione
123         massima del frame ethernet (unita' dati trasportata al livello 2). */
124         printf("Byte ricevuti: %d\n", tmp);
125     }
126     response[i] = 0; // Terminatore di stringa.
127
128     /* Stampa pagina web ricevuta. La risposta tornata dal server e' una
129     Full-Response e non una Simple-Response, ovvero il server utilizza
130     il protocollo HTTP 1.0. */
131     //printf("%s\n", response);
132
133     printf("\nDimensione pagina web: %d byte\n", i);
134
135     return 0;
136 }
```

4 HTTP 1.0

Nella precedente sezione abbiamo visto come creare un web client che si collega ai server di google. Abbiamo visto inoltre come inviare una richiesta utilizzando il primo protocollo web (HTTP 0.9). In questa sezione andiamo a studiare nel dettaglio il protocollo HTTP, ovvero il protocollo standard utilizzato a livello applicativo per la trasmissione di dati sul web.

Ad oggi il protocollo utilizzato maggiormente è HTTP 1.0 e HTTP 1.1, si sta iniziando ad utilizzare anche HTTP 2.0.

4.1 RFC 1945, HTTP 1.0

La RFC (*Request For Comments*) numero 1945 viene pubblicata nel 1996 e introduce per la prima volta il protocollo HTTP 1.0. Il documento riporta il seguente abstract: *"The Hypertext Transfer Protocol (HTTP) is an application-level protocol with the lightness and speed necessary for distributed, collaborative, hypermedia information systems. It is a generic, stateless, object-oriented protocol which can be used for many tasks, such as name servers and distributed object management systems, through extension of its request methods (commands). A feature of HTTP is the typing of data representation, allowing systems to be built independently of the data being transferred."* Il protocollo HTTP 0.9 permetteva soltanto di scaricare documenti attraverso link ipertestuali; nella versione 1.0 permette di sviluppare applicazioni ben più complesse.

4.2 La grammatica

La sezione 2 della RFC 1945 è dedicata alla grammatica del protocollo. In questa sezione troviamo tutti i costrutti utilizzati per descrivere il protocollo. Partiamo considerando un esempio: vediamo come scrivere la data in maniera corretta nel protocollo HTTP.

```

1  HTTP-date      = rfc1123-date | rfc850-date | asctime-date
2
3  rfc1123-date   = wkday "," SP date1 SP time SP "GMT"
4  rfc850-date    = weekday "," SP date2 SP time SP "GMT"
5  asctime-date  = wkday SP date3 SP time SP 4DIGIT
6
7  date1          = 2DIGIT SP month SP 4DIGIT
8                  ; day month year (e.g., 02 Jun 1982)
9  date2          = 2DIGIT "-" month "-" 2DIGIT
10                 ; day-month-year (e.g., 02-Jun-82)
11  date3          = month SP ( 2DIGIT | ( SP 1DIGIT ) )
12                 ; month day (e.g., Jun 2)
13
14  time           = 2DIGIT ":" 2DIGIT ":" 2DIGIT
15                 ; 00:00:00 - 23:59:59
16
17  wkday          = "Mon" | "Tue" | "Wed"
18                 | "Thu" | "Fri" | "Sat" | "Sun"
19
20  weekday        = "Monday" | "Tuesday" | "Wednesday"
21                 | "Thursday" | "Friday" | "Saturday" | "Sunday"
22
23  month          = "Jan" | "Feb" | "Mar" | "Apr"
24                 | "May" | "Jun" | "Jul" | "Aug"
25                 | "Sep" | "Oct" | "Nov" | "Dec"

```

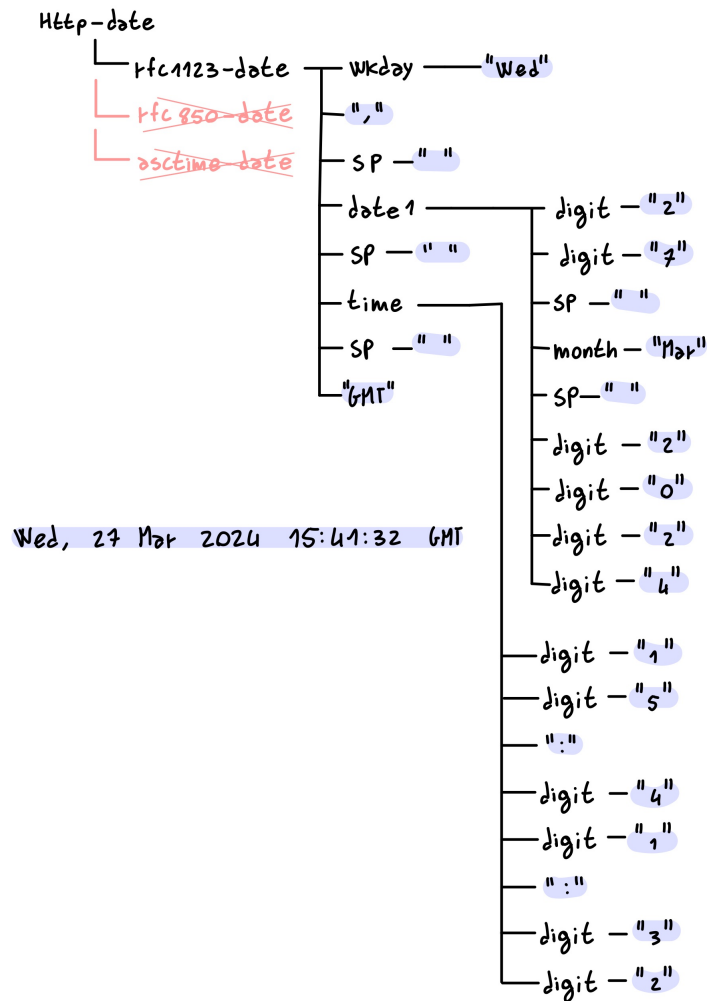


Figura 8: Esempio di applicazione della grammatica per costruire una data che rispetta il protocollo HTTP.

Alla radice troviamo il concetto astratto di data in HTTP; l'unione delle foglie forma una delle possibili stringhe che rappresentano una data. La grammatica ci permette di costruire strutture compatibili con il protocollo HTTP e da queste ricaviamo delle stringhe compatibili.

Definiamo alcuni costrutti della grammatica:

```

1 [2.1 Augmented BNF]
2
3 name = definition
4   The name of a rule is simply the name itself (without any
5   enclosing "<" and ">") and is separated from its definition by
6   the equal character "=".
7
8 "literal"

```

```

9      Quotation marks surround literal text. Unless stated otherwise,
10     the text is case-insensitive.
11
12     rule1 | rule2
13     Elements separated by a bar ("|") are alternatives (OR),
14     e.g., "yes | no" will accept yes or no.
15
16     (rule1 rule2)
17     Elements enclosed in parentheses are treated as a single
18     element. Thus, "(elem (foo | bar) elem)" allows the token
19     sequences "elem foo elem" and "elem bar elem".
20
21     *rule
22     The character "*" preceding an element indicates repetition. The
23     full form is "<n>*<m>element" indicating at least <n> and at
24     most <m> occurrences of element. Default values are 0 and
25     infinity so that "*(element)" allows any number, including zero;
26     "1*element" requires at least one; and "1*2element" allows one
27     or two.
28
29     [rule]
30     Square brackets enclose optional elements; "[foo bar]" is
31     equivalent to "*1(foo bar)".
32
33     N rule
34     Specific repetition: "<n>(element)" is equivalent to
35     "<n>*<n>(element)"; that is, exactly <n> occurrences of
36     (element). Thus 2DIGIT is a 2-digit number, and 3ALPHA is a
37     string of three alphabetic characters.
38
39     #rule
40     A construct "#" is defined, similar to "*", for defining lists
41     of elements. The full form is "<n>#<m>element" indicating at
42     least <n> and at most <m> elements, each separated by one or
43     more commas (",") and optional linear whitespace (LWS). This
44     makes the usual form of lists very easy.
45
46     ; comment
47     A semi-colon, set off some distance to the right of rule text,
48     starts a comment that continues to the end of line.
49
50     implied *LWS
51     The grammar described by this specification is word-based.
52     Except where noted otherwise, linear whitespace (LWS) can be
53     included between any two adjacent words (token or
54     quoted-string), and between adjacent tokens and delimiters
55     (tspecials), without changing the interpretation of a field.

```

4.3 HTTP Message

Nella sezione *HTTP Message* della RFC 1945 troviamo le specifiche sui messaggi che possono scambiarsi client e server.

Un messaggio HTTP consiste in una richiesta da parte del client al server e di una risposta del server al client.


```
1 [4.1 Message Types]
2
3 HTTP-message = Simple-Request      ; HTTP/0.9 messages
4                | Simple-Response
5                | Full-Request       ; HTTP/1.0 messages
6                | Full-Response
```

La richiesta e la risposta possono essere effettuate secondo il vecchio protocollo HTTP 0.9 oppure secondo il protocollo HTTP 1.0.

Simple-Request e Simple-Response Vengono definite come segue:

```
1 Simple-Request = "GET" SP Request-URI CRLF
2 Simple-Response = [ Entity-Body ]
```

Il *Request-URI* indica il path del file richiesto al server, URI¹ sta per *Uniform Resource Identifier*, infine il *CRLF* corrisponde a *Carriage Return Line Feed*. Nella risposta del server, l'*Entity-Body* corrisponde al contenuto del file richiesto nella request. Nota che l'*Entity-Body* nella risposta è facoltativo (facendo riferimento alla grammatica).

Full-Request Viene definita come segue:

```
1 Full-Request = Request-Line
2                *( General-Header
3                  | Request-Header
4                  | Entity-Header )
5                CRLF
6                [ Entity-Body ]
```

Il *CRLF* permette di dividere l'header dal body. Rispetto alla *Simple-Request*, nella *Full-Request* è possibile specificare un qualche tipo di header e anche body. In questo modo si riescono ad inviare dati al server².

```
1 Request-Line = Method SP Request-URI SP HTTP-Version CRLF
2
3 Method = "GET"
4         | "HEAD"
5         | "POST"
6         | extension-method
7
8 extension-method = token
9
10 HTTP-Version = "HTTP" "/" 1*DIGIT "." 1*DIGIT
```

Notiamo che *"GET"* fa parte dei metodi e in HTTP 1.0 non è l'unico, per esempio *"POST"* ci permette di inviare contenuti al server. Notiamo inoltre che va specificata la versione del protocollo utilizzato. Si tratta di un parametro fondamentale che permette al server di interpretare correttamente la richiesta che inviamo. Di prassi la versione del protocollo che si intende utilizzare viene sempre specificata come primo parametro, in questo caso è l'ultimo e ciò è stato fatto per mantenere la retrocompatibilità con il vecchio protocollo.

¹Il più noto URL è un sottoinsieme dell'URI.

²Per esempio quando si utilizza Gmail su browser è possibile allegare un documento.

Full-Response Definita come segue:

```

1 Full-Response = Status-Line
2                 *( General-Header
3                   | Response-Header
4                   | Entity-Header )
5                 CRLF
6                 [ Entity-Body ]

```

La struttura è molto simile alla *Full-Request*. La *Status-Line* specifica se la risposta è andata o meno a buon fine.

```

1 Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF
2
3 Status-Code = "200" ; OK
4              | "201" ; Created
5              ; ...
6              | "404" ; Not Found
7              ; ...
8
9 Reason-Phrase = *<TEXT, excluding CR, LF>

```

Nella risposta la versione del protocollo è il primo argomento, e quindi la retrocompatibilità non è stata mantenuta. Lo *Status-Code* fornisce il codice dell'errore, mentre la *Reason-Phrase* è una stringa che ci permette di interpretare l'errore³.

4.4 wc_HTTP_1.0.c

Riscriviamo il webclient che abbiamo realizzato nella precedente sezione in modo che sia conforme al protocollo HTTP 1.0.

```

1 #include <stdio.h>
2 #include <sys/socket.h>
3 #include <netinet/in.h>
4 #include <unistd.h>
5
6 /* Struttura di un header:
7    Nome | Valore
8 */
9 struct headers {
10     char* name;
11     char* value;
12 } h[100];
13
14 int main() {
15     // ### Creazione socket ###
16     int s;
17     s = socket(AF_INET, SOCK_STREAM, 0);
18
19     // ### Connect ###
20     struct sockaddr_in server;
21     server.sin_family = AF_INET;

```

³La Reason-Phrase è utile per noi umani, non per la macchina. Il fatto che ci sia non è scontato visto che comporta l'invio di una maggiore quantità di dati. Il protocollo HTTP 1.0, come tutti i **simple protocol**, segue questa filosofia che facilita il debugging. In HTTP 2.0 e 3.0 si sta abbandonando questo approccio in favore di migliori prestazioni.

```

22     server.sin_port = htons(80);
23
24     unsigned char* p = (unsigned char*) &server.sin_addr.s_addr;
25     p[0] = 216; p[1] = 58; p[2] = 209; p[3] = 36;
26
27     int tmp;
28     tmp = connect(s, (struct sockaddr*) &server, sizeof(struct sockaddr_in));
29     if (tmp == -1) {
30         perror("Connect fails");
31         return 1;
32     }
33
34     // ### Write ###
35     // Full-Request
36     write(s, "GET / HTTP/1.0\r\n\r\n", 18);
37
38     // ### HTTP parser ###
39     /* Vogliamo realizzare un parser che ci permetta di ricavare
40        solamente l'Entity-Body della Full-Response e che di conseguenza
41        scarti l'header.
42
43        Per realizzare un parser efficiente bisogna che questo lavori
44        mentre i dati stanno arrivando, non possiamo aspettare che
45        arrivino tutti prima di iniziare. */
46
47     /* Scarto l'Header. */
48     char hbuf[10000]; // Buffer header.
49
50     h[0].name = "Status line\0";
51     h[0].value = hbuf;
52
53     int i, j;
54     for (i = 0, j = 0; read(s, hbuf + i, 1); i++) {
55
56         // Setto il puntatore al Valore.
57         if (hbuf[i] == ':' && h[j].value == NULL) {
58             h[j].value = &hbuf[i+1];
59             hbuf[i] = 0; // Terminatore di stringa.
60         }
61
62         // Setto il puntatore al Nome o termino.
63         else if (i > 0 && hbuf[i] == '\n' && hbuf[i-1] == '\r') {
64             hbuf[i-1] = 0; // Terminatore di stringa.
65
66             /* Contollo se l'header e' terminato.
67                L'header termina quando ho un doppio 'CRLF'. */
68             if (h[j].name[0] == 0) {
69                 /* Se h[j].name[0] == 0 allora siamo sicuri che ci sono
70                    due 'CRLF' di fila e l'header e' terminato. */
71                 break;
72             }
73             // Setto il puntatore al Nome.
74             h[++j].name = &hbuf[i+1];
75         }
76     }
77     printf("Dimensione header: %d byte\n", i);

```

```
78
79 // Stampa header.
80 printf("\nContenuto Header:\n");
81 for (i = 0; i < j; i++) {
82     printf("%s:%s\n", h[i].name, h[i].value);
83 }
84
85 // Prelevo l'Entity Body.
86 const int dim = 100000;
87 char response[dim];
88 for (i = 0; tmp = read(s, response + i, (dim - 1) - i); i += tmp) ;
89 response[i] = 0; // Terminatore di stringa.
90 printf("\nDimensione Entity Body: %d byte\n", i);
91
92 // Stampa Entity Body.
93 printf("\nContenuto Entity Body:\n%s\n", response);
94
95 return 0;
96 }
```

5 HTTP 1.1

Abbiamo visto come realizzare un webclient che utilizza il protocollo HTTP 1.0. Ora facciamo un passo avanti e analizziamo quali sono le migliorie che sono state fatte nella versione 1.1.

5.1 Three-way handshake

Nella sezione precedente abbiamo introdotto il protocollo HTTP 1.0 e abbiamo realizzato un webclient che si collega ai server di Google ed invia una richiesta. Durante l'apertura della connessione, quello che accade in pratica è uno scambio di comunicazioni tra client e server: si parla di **three-way handshake**. La three-way handshake permette di instaurare una connessione TCP sicura tra client e server. Tale procedura è composta da tre passaggi che non analizziamo. Ci basta sapere che si tratta di tre deboli segnali che viaggiano attraverso la rete e che di fatto servono solo ad instaurare la comunicazione: non portano dati. Terminata questa fase il client fa la richiesta e il server risponde inviando i dati.

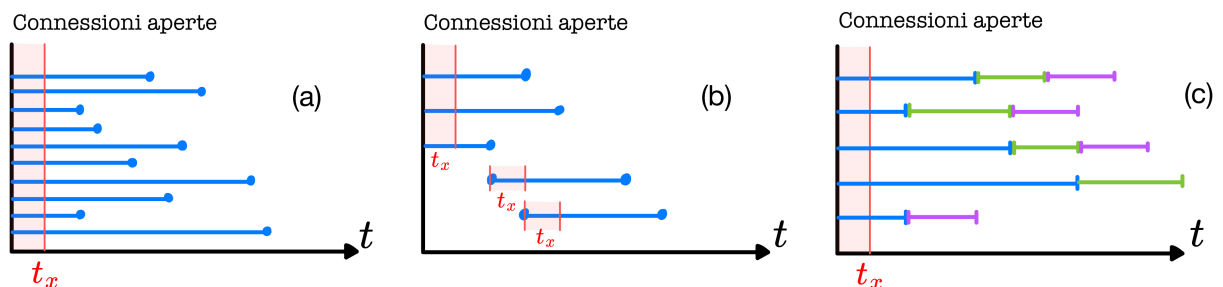


Figura 9: (a) Il numero di connessioni aperte è illimitato; t_x è il tempo perso a causa della creazione dell'instaurazione della connessione (dovuto alla latenza). Ogni connessione gestisce un'unica richiesta, dopodiché viene chiusa. Il problema è il numero di connessioni aperte. (b) Il numero di connessioni aperte contemporaneamente è limitato a 3; le richieste successive vengono ritardate. Sussiste il problema dello slow start ad ogni nuova richiesta. (c) Ogni connessione permette molteplici richieste (connection keep alive), in questo modo si soffre del fenomeno dello slow start solamente all'inizio.

5.2 Connection keep alive

Effettuando una richiesta del tipo *GET /pagina.html* in HTTP 1.1, oltre al testo della pagina, vengono scaricati anche tutti gli oggetti che sono presenti e ognuno di questi scatena a sua volta N richieste (*GET*) che ritornano il contenuto. Il problema è che utilizzando il protocollo HTTP 1.0 ogni *GET* corrisponde ad una nuova apertura di connessione e queste, lato server, diventano molto costose considerando la grossa mole di richieste che devono gestire.

Una prima soluzione è stata quella di porre un limite al numero di connessioni aperte contemporaneamente, ma non bastava, bisognava trovare una soluzione migliore. Il protocollo HTTP 1.1 nasce con l'intento di migliorare le prestazioni rispetto ad HTTP 1.0. L'idea è quella di aprire una singola connessione che non si interrompe dopo l'invio della risposta del server ma invece rimane aperta per gestire ulteriori richieste da parte del client. Il comportamento che abbiamo appena descritto prende il nome di **connection keep alive** ed è utilizzato di default nel protocollo HTTP 1.1. Sfruttando il connection keep alive, l'inefficienza dovuta all'apertura della connessione si ha solamente all'inizio (**slow start**).

Un'altra miglioria introdotta da HTTP 1.1 è l'efficiente utilizzo della connessione *full-duplex*: è possibile inviare molteplici richieste e aspettare le conseguenti risposte aprendo una singola comunicazione client server.

5.3 Chunked Transfer Coding e Content-Length

Dato che la connessione non viene chiusa una volta che il server ha finito di inviare i dati richiesti, bisogna utilizzare un altro meccanismo per comunicare al client che l'invio è terminato. Due possibili soluzioni sono il **chunked transfer coding** e il **content-length**.

Il content-length è un parametro che si trova nell'header della risposta inviata dal server che specifica la lunghezza dell'entity body inviato. Sfruttando questo parametro si può facilmente riconoscere la fine di un'entity body.

In alcuni casi il content-length non è disponibile. Questa situazione si verifica ad esempio quando il file cercato non è staticamente presente nel server ma deve essere generato in backend. In questi casi, aspettare che il dato sia prodotto e poi procedere con l'invio sfruttando il content-length, è uno spreco di risorse (la linea rimante inutilizzata). Un approccio più efficiente è quello di utilizzare il chunked transfer coding. Dalla sezione 3.6.1 dell'RFC numero 2616 si legge che:

[3.6.1 Chunked Transfer Coding]

The chunked encoding modifies the body of a message in order to transfer it as a series of chunks, each with its own size indicator, followed by an OPTIONAL trailer containing entity-header fields. This allows dynamically produced content to be transferred along with the information necessary for the recipient to verify that it has received the full message.

```
Chunked-Body = *chunk
               last-chunk
               trailer
               CRLF
```

```
chunk         = chunk-size [ chunk-extension ] CRLF
               chunk-data CRLF
```

```
chunk-size    = 1*HEX
```

```
last-chunk    = 1*("0") [ chunk-extension ] CRLF
```

```
chunk-extension= *( ";" chunk-ext-name [ "=" chunk-ext-val ] )
```

```
chunk-ext-name = token
```

```
chunk-ext-val  = token | quoted-string
```

```
chunk-data     = chunk-size(OCTET)
```

```
trailer        = *(entity-header CRLF)
```

The chunk-size field is a string of hex digits indicating the size of the chunk. The chunked encoding is ended by any chunk whose size is zero, followed by the trailer, which is terminated by an empty line.

The trailer allows the sender to include additional HTTP header fields at the end of the message. The Trailer header field can be used to indicate which header fields are included in a trailer (see section 14.40).

In sostanza nel chunked transfer coding viene inviato un certo quantitativo di byte per volta. La dimensione del chunk non è fissa e viene specificata di volta in volta; quando la dimensione è uguale a zero, allora significa che la response è terminata.

5.4 wc_HTTP_1_1.c

```
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <netinet/in.h>
4  #include <unistd.h>
5  #include <string.h>
6  #include <stdlib.h>
7
8  /* Struttura di un header:
9      Nome | Valore
10 */
11 struct headers {
12     char* name;
13     char* value;
14 } h[100];
15
16 struct chunk {
17     char* size;
18     char* data;
19 } c[100];
20
21 int main() {
22     int s;
23     s = socket(AF_INET, SOCK_STREAM, 0);
24
25     struct sockaddr_in server;
26     server.sin_family = AF_INET;
27     server.sin_port = htons(80);
28
29     unsigned char* p = (unsigned char*) &server.sin_addr.s_addr;
30     p[0] = 216; p[1] = 58; p[2] = 209; p[3] = 36;
31
32     int tmp;
33     tmp = connect(s, (struct sockaddr*) &server, sizeof(struct sockaddr_in));
34     if (tmp == -1) {
35         perror("Connect fails");
36         return 1;
37     }
38
39     // Request in HTTP 1.1.
40     //char* request = "GET /ppp HTTP/1.1\r\n\r\n"; // Content-Length: x byte
41     char* request = "GET / HTTP/1.1\r\n\r\n"; // Transfer-Encoding: chunked
42     write(s, request, strlen(request));
43
44     // ### HTTP parser ###
45     // Scarto l'Header.
46     char hbuf[10000]; // Buffer header.
47
48     h[0].name = "Status line";
49     h[0].value = hbuf;
```

```

50
51 int i, j;
52 for (i = 0, j = 0; read(s, hbuf + i, 1); i++) {
53
54     // Setto il puntatore al Valore.
55     if (hbuf[i] == ':' && h[j].value == NULL) {
56         h[j].value = &hbuf[i+1];
57         hbuf[i] = 0; // Terminatore di stringa.
58     }
59
60     // Setto il puntatore al Nome o termino.
61     else if (i != 0 && hbuf[i] == '\n' && hbuf[i-1] == '\r') {
62         hbuf[i-1] = 0; // Terminatore di stringa.
63
64         /* Controllo se l'header e' terminato.
65          L'header termina quando ho un doppio 'CRLF'. */
66         if (h[j].name[0] == 0) {
67             break;
68         }
69         // Setto il puntatore al Nome.
70         h[++j].name = &hbuf[i+1];
71     }
72 }
73 printf("Dimensione header: %d byte\n", i);
74
75 // Stampa header e cotrollo se e' presente Content-Length.
76 int len = 0;
77 printf("\nContenuto Header:\n");
78 for (i = 0; i < j; i++) {
79     printf("%s:%s\n", h[i].name, h[i].value);
80
81     // Prelevo la lunghezza entity body.
82     if (!strcmp(h[i].name, "Content-Length")) {
83         len = atoi(h[i].value);
84     }
85 }
86
87
88 // Se e' stato specificato Content-Lenght.
89 if (len) {
90     char response[100000];
91
92     // Salvo l'Entity Body.
93     for (i = 0; tmp = read(s, response + i, len - i); i += tmp) ;
94     response[i] = 0; // Terminatore di stringa.
95
96     // Stampa contenuto Entity Body.
97     printf("\nContenuto Entity Body:\n%s\n", response);
98 }
99 // Se viene utilizzato il chunk transfer.
100 else {
101     char chunked_body[100000];
102
103     c[0].size = chunked_body;
104     for (i = 0, j = 0; tmp = read(s, chunked_body + i, 1); i++) {
105         if (chunked_body[i] == '\n' && chunked_body[i-1] == '\r') {

```



```
106         chunked_body[i-1] = 0;
107
108         if (c[j].size == &chunked_body[i+1]) continue;
109
110         c[j].data = &chunked_body[i+1];
111
112         int s = (int) strtol(c[j].size, NULL, 16);
113         printf("\nsize = %d\n", s);
114         printf("sizeHex = %s\n", c[j].size);
115         if (!s) break;
116
117         c[++j].size = &chunked_body[i+1 + s+2];
118     }
119 }
120
121 // Stampa contenuto Entity Body.
122 int dim_page = 0;
123 for (i = 0; i < j; i++) {
124     printf("%s", c[i].data);
125     dim_page += (int) strtol(c[i].size, NULL, 16);
126 }
127 printf("\ndim_page = %d\n", dim_page);
128
129 }
130
131 return 0;
132 }
```

6 Cenni su HTTP 2.0

6.1 Framing dello stream e richieste/risposte interleaved

In HTTP 1.1 abbiamo visto che è possibile stabilire un'unica connessione tra client e server ed inviare molteplici richieste. La principale differenza introdotta in HTTP 2.0 (RFC 7945) è che le richieste vengono spezzettate in pacchetti (**frame**): non sono più contigue e sequenziali come nella versione precedente ma possono essere spezzettate ed inviate in parallelo al server (**interleaved**). In maniera simile il server risponde inviando frame di risposte in parallelo.

Grazie all'interleaving il server può inviare frame di risposte che richiedono meno tempo per essere gestite (es: invio di dati statici) e inviare intanto anche frame di risposte più pesanti, che richiedono più passaggi computazionali (es: invio di dati dinamici) evitando in questo modo i tempi morti (presenti in l'HTTP 1.1).

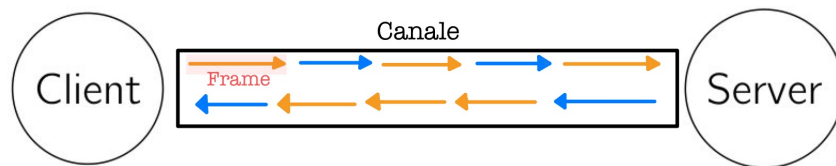


Figura 10: HTTP 2.0: le richieste e le risposte sono interleaved.

7 Cenni su HTTP 3.0

7.1 QUIC over UDP

Siccome in HTTP 2.0 le risposte e le richieste vengono inviate in pacchetti, si è pensato di passare ad utilizzare il servizio UDP al posto di TCP (SOCK_DGRAM). Il vantaggio è che la connessione tra client e server viene gestita in maniera più efficiente: non sono più necessari i 3 handshake come nel caso di TCP; non viene creato un canale di comunicazione e ogni pacchetto viaggia per conto suo. Lo svantaggio principale è che UDP non gestisce la congestione dei pacchetti: non viene gestita a livello kernel come in TCP, deve essere implementata a livello applicativo. In altre parole in HTTP 3.0 l'efficienza della gestione di congestione dei pacchetti dipende da chi sviluppa l'applicazione.

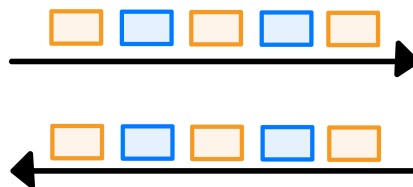


Figura 11: HTTP 3.0.

8 Realizzare un webserver

Nel modello client-server, il server rimane in attesa di servire le richieste provenienti dai client. Le chiamate a sistema principali per lo sviluppo del server web sono le seguenti:

- `bind`: associa il socket all'indirizzo ip della macchina e ad un port.
- `listen`: predispone il socket a ricevere richieste di connessione (**socket passivo**, viceversa la `connect` definisce un **socket attivo**).
- `accept`: blocca il programma finché non arriva una richiesta e ritorna un nuovo socket specifico per quella connessione.

Le chiamate di sistema `accept` e `listen` sono specifiche per il server; la chiamata `bind` veniva effettuata (implicitamente) dalla chiamata a sistema `connect` che abbiamo utilizzato per il webclient. `connect` effettua un `bind` implicito e sceglie il primo port libero, alternativamente è possibile effettuare una chiamata esplicita e scegliere il port (prima di chiamare `connect`).

8.1 Bind(2)

Il primo passo è quello di associare il socket (che abbiamo creato con la system call `socket`) ad uno specifico port. Per farlo utilizziamo la system call '`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`'. Nota che il port (che deve essere well known) è associato ad un unico processo all'interno della macchina; un socket ha un numero (file descriptor) unico all'interno del processo ma più socket possono essere associati allo stesso port. Analizziamo i parametri della `bind`:

- `int sockfd`: file descriptor socket;
- `const struct sockaddr* addr`: contiene indirizzo (locale del server) e il port;
- `size_t sockaddr_in addrlen`: `sizeof` di `addr`.

8.2 Listen(2)

Ora che abbiamo associato il socket al port, possiamo rendere il socket passivo. Per farlo utilizziamo la system call '`int listen(int sockfd, int backlog)`'. Analizziamo i parametri:

- `int sockfd`: file descriptor socket;
- `int backlog`: fissiamo la dimensione della coda (FIFO) delle richieste pendenti. Se la coda è piena, le richieste successive vengono rifiutate (errore *Connection refused*).

8.3 Accept(2)

Attraverso la system call '`int accept(int sockfd, struct sockaddr *_Nullable restrict addr, socklen_t *_Nullable restrict addrlen)`' possiamo accettare le richieste entranti. `accept` ritorna il file descriptor di nuovo socket collegato al port specificato nella chiamata di `bind`. Questo nuovo socket è attivo, non passivo. Il socket passivo che abbiamo creato con la prima chiamata di `socket` rimane sempre in ascolto e il suo unico compito è quello di creare nuovi socket attivi associati al suo stesso port. I socket attivi creati gestiscono le richieste in entrata. Analizziamo i parametri:

- `int sockfd`: file descriptor socket passivo.

- `struct sockaddr *_Nullable restrict addr`: passiamo l'indirizzo di una variabile di tipo `struct sockaddr_in` nel quale verranno scritte le informazioni relative al client che ha inviato la richiesta (l'indirizzo IP e il suo port).
- `socklen_t *_Nullable restrict addrlen`: indirizzo della variabile che contiene la `sizeof` di `addr`.

8.4 ws.c

```

1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <errno.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <string.h>
7  #include <stdlib.h>
8
9  /* Struttura di un header:
10     Nome | Valore
11  */
12  struct headers {
13     char* name;
14     char* value;
15 } h[100];
16
17 int main(void) {
18     char request[3001];
19     char response[3001];
20
21     int s = socket(AF_INET, SOCK_STREAM, 0);
22     if (s == -1) {
23         perror("Socket fails");
24         return 1;
25     }
26
27     struct sockaddr_in srvaddr;
28     srvaddr.sin_family = AF_INET;
29     srvaddr.sin_port = htons(9010);
30     srvaddr.sin_addr.s_addr = 0; // 0 => indirizzo della macchina.
31
32
33     /* Evita errori generati dalla bind su indirizzi utilizzati
34        recentemente */
35     int yes = 1;
36     int t = setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
37     if (t == -1) {
38         perror("Setsockopt fails");
39         return 1;
40     }
41
42     // ### Bind ###
43     t = bind(s, (struct sockaddr*) &srvaddr, sizeof(struct sockaddr_in));
44     if (t == -1) {
45         perror("Bind fails");
46         return 1;

```

```
47     }
48
49 // ### Listen ###
50 /* Rendo il socket passivo e fisso la dimensione della coda delle richieste
51    pendenti a 5. */
52 t = listen(s, 5);
53 if (t == -1) {
54     perror("Listen fails");
55     return 1;
56 }
57
58 // ### Accept ###
59 /* Accetto le richieste. */
60 struct sockaddr_in remote;
61 int len = sizeof(struct sockaddr);
62
63 int s2; // File descriptor del socket tornato da accept.
64 while (1) {
65     close(s2);
66     s2 = accept(s, (struct sockaddr*) &remote, &len);
67
68     /* Faccio gestire ai processi figli le richieste */
69     if (fork()) continue;
70
71     if (s2 == -1) {
72         perror("Accept fails");
73         exit(1);
74     }
75
76     // Stampo informazioni client.
77     printf("\nClient port = %u, addr = %u\n\n", htons(remote.sin_port),
78           htons(remote.sin_addr.s_addr));
79
80 // ### Parsing Header ###
81     char hbuf[10000]; // Buffer header.
82
83     h[0].name = "Request Line";
84     h[0].value = hbuf;
85
86     int i, j;
87     for (i = 0, j = 0; read(s2, hbuf + i, 1); i++) {
88
89         // Setto il puntatore al Valore.
90         if (hbuf[i] == ':' && h[j].value == NULL) {
91             h[j].value = &hbuf[i+1];
92             hbuf[i] = 0; // Terminatore di stringa.
93         }
94
95         // Setto il puntatore al Nome o termino.
96         else if (i > 0 && hbuf[i] == '\n' && hbuf[i-1] == '\r') {
97             hbuf[i-1] = 0; // Terminatore di stringa.
98
99             /* Controllo se l'header e' terminato.
100                L'header termina quando ho un doppio 'CRLF'. */
101             if (h[j].name[0] == 0) {
102                 break;
103             }
104         }
105     }
106 }
```

```

102         }
103         // Setto il puntatore al Nome.
104         h[++j].name = &hbuf[i+1];
105     }
106 }
107
108 // Stampa header.
109 for (i = 0; i < j; i++) {
110     printf("%s:%s\n", h[i].name, h[i].value);
111 }
112 printf("\n");
113
114 // ### Parsing Request-Line ###
115 // Request-Line = Method SP Request-URI SP HTTP-Version CRLF
116 char *method, *filename, *http_version;
117 method = h[0].value;
118 for (i = 0; h[0].value[i] != ' '; i++) ;
119 h[0].value[i] = 0;
120 i += 1;
121
122 filename = h[0].value + i;
123 for ( ; h[0].value[i] != ' '; i++) ;
124 h[0].value[i] = 0;
125 i += 1;
126
127 http_version = h[0].value + i; // Il terminatore e' stato inserito
128                                // (al posto di '\r') nel parser
129                                // dell'header.
130
131 // ### Response ###
132 /* Gestione esecuzione comando.
133    Controllo se il filename richiesto comincia con il path
134    speciale (che abbiamo scelto) "/exec". */
135 if (!strcmp("/exec/", filename, 6)) {
136     // Prelevo il comando da eseguire.
137     char command[100];
138     sprintf(command, "%s > out.txt", filename + 6);
139     system(command);
140     filename = "/out.txt"; // Invio al client l'output del comando.
141 }
142
143 /* Full-Response: gestione file richiesto. */
144 FILE* fin = fopen(filename + 1, "rt"); // Il '+1' per rimuovere il path
145                                         // relativo.
146 if (fin == NULL) {
147     sprintf(response, "HTTP/1.1 404 NOT
148                     FOUND\r\nConnection:close\r\n\r\n<html><h1>404 File %s
149                     found</h1></html>", filename);
150     write(s2, response, strlen(response));
151     close(s2);
152     exit(1);
153 }
154
155 sprintf(response, "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Type:
156     text/html\r\nDate: Sat, 27 Apr 2024 10:27:00 GMT\r\n\r\n");
157 write(s2, response, strlen(response));

```

```
155
156     char entity[1000];
157     while (!feof(fin)) {
158         fread(entity, 1, 1000, fin);
159         write(s2, entity, 1000);
160     }
161
162     exit(0); // Per far terminare il processo figlio.
163 }
164
165 return 0;
166 }
```

9 Il caching

9.1 Il principio del caching

Il caching in HTTP si basa sullo stesso principio su cui si basa il funzionamento della cache nelle CPU. Quando effettuiamo una richiesta al server per scaricare un contenuto, quel contenuto lo dobbiamo veramente scaricare o ce lo abbiamo già? Il caching svolge le due seguenti funzioni:

- Salvataggio in locale dei contenuti scaricati dal client: viene allocato dello spazio su disco per salvare i contenuti delle richieste effettuate in precedenza dal client, organizzato per URL.
- Permette di decidere se usare la copia salvata in cache oppure scaricare nuovamente il contenuto (perché è stato modificato).

Il problema è la decisione: come faccio a sapere se devo ricaricare la pagina o posso invece usare la copia salvata in locale? Esiste un'informazione che ci permette di prendere una decisione sicura? Il protocollo HTTP fornisce le informazioni necessarie per poter prendere una decisione. Nella prossima sottosezione vediamo i meccanismi che permettono di effettuare questa decisione.

9.2 I meccanismi del caching

Metodo 1: l'header Expires Nella sezione 10.7 della RFC 1945 (HTTP 1.0) si legge che:

```

1 [10.7 Expires]
2
3 The Expires entity-header field gives the date/time after which the
4 entity should be considered stale. This allows information providers
5 to suggest the volatility of the resource, or a date after which the
6 information may no longer be valid. Applications must not cache this
7 entity beyond the date given. The presence of an Expires field does
8 not imply that the original resource will change or cease to exist
9 at, before, or after that time. However, information providers that
10 know or even suspect that a resource will change by a certain date
11 should include an Expires header with that date. The format is an
12 absolute date and time as defined by HTTP-date in Section 3.3.
13
14 Expires      = "Expires" ":" HTTP-date
15
16 An example of its use is
17
18 Expires: Thu, 01 Dec 1994 16:00:00 GMT
19
20 If the date given is equal to or earlier than the value of the Date
21 header, the recipient must not cache the enclosed entity. If a
22 resource is dynamic by nature, as is the case with many data-
23 producing processes, entities from that resource should be given an
24 appropriate Expires value which reflects that dynamism.
25
26 The Expires field cannot be used to force a user agent to refresh its
27 display or reload a resource; its semantics apply only to caching
28 mechanisms, and such mechanisms need only check a resource's
29 expiration status when a new request for that resource is initiated.
30
```


User agents often have history mechanisms, such as "Back" buttons and history lists, which can be used to redisplay an entity retrieved earlier in a session. By default, the Expires field does not apply to history mechanisms. If the entity is still in storage, a history mechanism should display it even if the entity has expired, unless the user has specifically configured the agent to refresh expired history documents.

Note: Applications are encouraged to be tolerant of bad or misinformed implementations of the Expires header. A value of zero (0) or an invalid date format should be considered equivalent to an "expires immediately." Although these values are not legitimate for HTTP/1.0, a robust implementation is always desirable.

Consideriamo un esempio in cui il client effettua una richiesta al server e questo, nella risposta, inserisce i due seguenti header:

```
Expires: Mon, 22 Apr 2024 18:00:00 GMT
Date:    Mon, 22 Apr 2024 16:00:00 GMT
```

Al file scaricato viene aggiunta una informazione aggiuntiva che indica la sua **scadenza** e successivamente viene salvato nella cache. La scadenza del file è data da:

$$\text{Scadenza} = \text{now()} + \text{Expires} - \text{Date}$$

dove `now()` è una funzione che ritorna la data e l'ora corrente che viene invocata appena ricevuta la risposta del server. Se il client effettua una stessa richiesta al tempo $T < \text{Scadenza}$, allora il file viene recuperato dalla cache. Altrimenti ($T \geq \text{Scadenza}$) deve essere inoltrata una nuova richiesta al server.

Metodo 2: l'header If-Modified-Since Dalla RFC 1945 si legge che:

[10.9 If-Modified-Since]

The If-Modified-Since request-header field is used with the GET method to make it conditional: if the requested resource has not been modified since the time specified in this field, a copy of the resource will not be returned from the server; instead, a 304 (not modified) response will be returned without any Entity-Body.

If-Modified-Since = "If-Modified-Since" ":" HTTP-date

An example of the field is:

If-Modified-Since: Sat, 29 Oct 1994 19:43:31 GMT

A conditional GET method requests that the identified resource be transferred only if it has been modified since the date given by the If-Modified-Since header. The algorithm for determining this includes the following cases:

- a) If the request would normally result in anything other than a 200 (ok) status, or if the passed If-Modified-Since date is invalid, the response is exactly the same as for a normal GET. A date which is later than the server's current time is invalid.

```

25
26     b) If the resource has been modified since the
27         If-Modified-Since date, the response is exactly the same as
28         for a normal GET.
29
30     c) If the resource has not been modified since a valid
31         If-Modified-Since date, the server shall return a 304 (not
32         modified) response.
33
34     The purpose of this feature is to allow efficient updates of cached
35     information with a minimum amount of transaction overhead.

```

In questo caso, il client inserisce tra gli header della richiesta:

```

1     If-Modified-Since: Mon, 8 Apr 2024 15:00:00 GMT
2     Date:               Mon, 22 Apr 2024 18:00:00 GMT

```

dove *If-Modified-Since* è la data di salvataggio presente nel file salvato in cache. Il server, una volta ricevuta la richiesta, andrà a verificare se

$$\text{Data ultima modifica file richiesto} \leq \text{now()} + \text{If Modified Since} - \text{Date}$$

dove *Data ultima modifica file richiesto* è riferita al file memorizzato nel server. Se questo è il caso, allora significa che il client ha una versione aggiornata del file, quindi il server risponderà inviando

```

1     HTTP/1.1 304 Not Modified

```

in modo da notificare il client che può utilizzare la versione del file che è salvata in cache. Se invece

$$\text{Data ultima modifica file richiesto} > \text{now()} + \text{If Modified Since} - \text{Date}$$

allora il server risponderà inviando il contenuto del file richiesto (Entity-Body).

Valori speciali Esistono dei valori speciali che permettono al server di comunicare al client che il file inviato non deve essere salvato nella cache (utile per esempio se la pagina richiesta è dinamica). In questi casi possiamo trovare nella risposta uno dei due seguenti header:

```

1     Expires: -1
2     Pragma: no-cache

```

Metodo 3: l'header Last-Modified Dalla RFC 1945 si legge che:

```

1     [10.10 Last-Modified]
2
3     The Last-Modified entity-header field indicates the date and time at
4     which the sender believes the resource was last modified. The exact
5     semantics of this field are defined in terms of how the recipient
6     should interpret it: if the recipient has a copy of this resource
7     which is older than the date given by the Last-Modified field, that
8     copy should be considered stale.
9
10    Last-Modified = "Last-Modified" ":" HTTP-date
11

```

12 An example of its use is

13
14 Last-Modified: Tue, 15 Nov 1994 12:45:26 GMT

15
16 The exact meaning of this header field depends on the implementation
17 of the sender and the nature of the original resource. For files, it
18 may be just the file system last-modified time. For entities with
19 dynamically included parts, it may be the most recent of the set of
20 last-modify times for its component parts. For database gateways, it
21 may be the last-update timestamp of the record. For virtual objects,
22 it may be the last time the internal state changed.

23
24 An origin server must not send a Last-Modified date which is later
25 than the server's time of message origination. In such cases, where
26 the resource's last modification would indicate some time in the
27 future, the server must replace that date with the message
28 origination date.

Sfruttando questo metodo, il client non effettua un *GET* ma un *HEAD*:

1 HEAD <Risorsa> HTTP/1.1

Il server risponderà con

1 HTTP/1.1 200 Ok
2 Last-Modified: Mon, 8 Apr 2024 16:30:00 GMT
3 Date: Mon, 22 Apr 2024 18:00:00 GMT

Il client allora andrà a verificare se

$$\text{Data salvataggio file} > \text{now()} + \text{Last Modified} - \text{Date}$$

dove *Data salvataggio file* indica la data in cui il file è stato salvato nella cache. Se ci troviamo in questo caso, allora il client deciderà di utilizzare il file salvato in cache. In caso contrario effettuerà una nuova richiesta (Full-Request) per scaricare il file aggiornato.

9.3 Confronto tra i meccanismi di caching

Abbiamo visto tre meccanismi con il quale HTTP implementa il caching. Vediamo in quali casi conviene utilizzare i vari metodi:

- **Metodo 1 (Expires):** si tratta del metodo più semplice che non richiede nessuna richiesta aggiuntiva o calcolo particolare. Il problema principale è che spesso è difficile dare una scadenza precisa. Nei casi in cui la scadenza è esatta (es: lettura di una temperatura ad istanti prestabiliti), allora questo è il miglior metodo. In generale è il miglior metodo se il file viene aggiornato molto frequentemente ed ad istanti di tempo noti.
- **Metodo 2 (If-Modified-Since):** metodo utilizzato quando non si sa con esattezza quando il file verrà aggiornato: i cambiamenti non sono predeterminati. Se il client effettua una richiesta e il server risponde comunicando che il file non è stato modificato, allora la richiesta è stata effettuata per niente. Questa è la penalità che si paga utilizzando questo metodo. In generale il metodo 2 viene utilizzato nei casi in cui gli aggiornamenti del file sono frequenti ma non temporizzati.

- **Metodo 3 (Last-Modified):** rispetto al metodo precedente, in questo caso, il server non deve effettuare nessun controllo sulle date di ultima modifica, è infatti il client ad occuparsene. Inoltre non viene effettuata una *GET* ma una *HEAD* in cui viene richiesta solo la data di ultima modifica. Se le modifiche al file sono poco frequenti, allora conviene utilizzare questo metodo: nei pochi casi in cui il file viene modificato viene effettuata anche una *GET*.

10 L'autenticazione

Il protocollo HTTP è **stateless**: le richieste effettuate per scaricare una pagina web non condividono nessuna informazione. Nella sezione precedente abbiamo visto come memorizzare uno stato (la data) e sfruttarlo per realizzare diversi meccanismi di caching. Anche l'autenticazione ha bisogno di un stato per funzionare. Nelle successive sottosezioni andremo a vedere come sia possibile memorizzare uno stato che permetta di realizzare il meccanismo dell'autenticazione.

10.1 Il principio dell'autenticazione

Vogliamo effettuare una richiesta di una pagina che non è accessibile a tutti gli utenti ma solamente ad un gruppo ristretto (detto **Realm**). L'utente dunque necessita di un modo per potersi identificare.

10.2 Basic Authentication Scheme

Nei casi in cui la pagina richiesta dall'utente necessita dell'identificazione, il server risponde con quella che viene chiamata **challenge**:

```
1 HTTP/1.1 401 Unauthorized
2 WWW-Authenticate: Basic realm="NomeRealm"
```

Il client (browser), ricevuta la challenge, fa visualizzare un pop-up in cui viene richiesto all'utente di identificarsi inserendo nei campi mostrati il nome utente e la password. Una volta inseriti i valori nei campi richiesti, il client andrà ad effettuare una nuova richiesta (sempre per la stessa risorsa) in cui inserirà i parametri di autenticazione dell'utente negli header:

```
1 GET <Risorsa>
2 Authorization: Basic <utente:password in base64>
```

Infine il server andrà a verificare se l'utente specificato appartiene al Realm e nel caso invierà la risorsa richiesta al client.

Abbiamo appena descritto come funziona il **Basic Authentication Scheme** che si trova nella RFC 1945 sotto la sezione 11.1.

```
1 [11.1 Basic Authentication Scheme]
2
3 The "basic" authentication scheme is based on the model that the user
4 agent must authenticate itself with a user-ID and a password for each
5 realm. The realm value should be considered an opaque string which
6 can only be compared for equality with other realms on that server.
7 The server will authorize the request only if it can validate the
8 user-ID and password for the protection space of the Request-URI.
9 There are no optional authentication parameters.
10
11 Upon receipt of an unauthorized request for a URI within the
12 protection space, the server should respond with a challenge like the
13 following:
14
15     WWW-Authenticate: Basic realm="WallyWorld"
16
17 where "WallyWorld" is the string assigned by the server to identify
18 the protection space of the Request-URI.
19
```

To receive authorization, the client sends the user-ID and password, separated by a single colon (":") character, within a base64 [5] encoded string in the credentials.

```
basic-credentials = "Basic" SP basic-cookie

basic-cookie      = <base64 encoding of userid-password,
                    except not limited to 76 char/line>
userid-password = [ token ] ":" *TEXT
```

If the user agent wishes to send the user-ID "Aladdin" and password "open sesame", it would use the following header field:

```
Authorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==
```

The basic authentication scheme is a non-secure method of filtering unauthorized access to resources on an HTTP server. It is based on the assumption that the connection between the client and the server can be regarded as a trusted carrier. As this is not generally true on an open network, the basic authentication scheme should be used accordingly. In spite of this, clients should implement the scheme in order to communicate with servers that use it.

10.3 La codifica base64

Nella precedente sottosezione abbiamo visto che il client, per autenticarsi, invia una stringa del tipo *utente:password* codificata in *base64*. La codifica base64 non è una chiave segreta ma permette semplicemente di codificare delle sequenze di bit in stringhe di testo ASCII. Si tratta di una codifica molto utile che è compatibile con tutti i protocolli testuali (come HTTP⁴). La codifica base64 si basa sul fatto che

$$\underbrace{8}_{\text{bit}} \times \underbrace{3}_{\text{byte}} = \underbrace{6}_{\text{bit}} \times \underbrace{4}_{\text{simboli base64}}$$

in altre parole, ogni tripletta di byte può essere codificata con 4 simboli base64. Dunque la lunghezza della sequenza codificata è maggiore rispetto alla sequenza di partenza.

	Value	Encoding	Value	Encoding	Value	Encoding	Value	Encoding
1								
2	0	A	17	R	34	i	51	z
3	1	B	18	S	35	j	52	0
4	2	C	19	T	36	k	53	1
5	3	D	20	U	37	l	54	2
6	4	E	21	V	38	m	55	3
7	5	F	22	W	39	n	56	4
8	6	G	23	X	40	o	57	5
9	7	H	24	Y	41	p	58	6
10	8	I	25	Z	42	q	59	7
11	9	J	26	a	43	r	60	8
12	10	K	27	b	44	s	61	9
13	11	L	28	c	45	t	62	+
14	12	M	29	d	46	u	63	/
15	13	N	30	e	47	v		
16	14	O	31	f	48	w	(pad)	=

⁴La codifica base64 viene utilizzata anche per l'invio di file binari (es: pdf) nelle e-mail.

17	15 P	32 g	49 x	
18	16 Q	33 h	50 y	[The Base64 Alphabet]

Consideriamo la seguente sequenza di 3 byte:

00010101 10101111 11110111

dividiamoli in 4 gruppi da 6 bit:

000101 011010 111111 110111

convertiamo i bit nei simboli base64 corrispondenti:

F a / 3

Nel caso in cui la stringa da codificare ha la lunghezza L che non è un multiplo di 3, allora:

- se $L \bmod 3 = 1$, allora viene aggiunto un '=' (pad) in coda alla codifica;
- se $L \bmod 3 = 2$, allora viene aggiunto un doppio '=' (pad) in coda alla codifica.

Per codificare delle stringhe in base64 si può utilizzare il comando `base64` su una shell unix, aggiungendo il parametro `-d` si ottiene la decodifica.

10.4 Basic-cookie

Il basic-cookie che troviamo nella RFC 1945 ha le stesse caratteristiche dei tanto famosi cookies. Il basic-cookie è la codifica base64 della stringa del nome utente e della password. Per spiegare l'utilità del basic-cookie consideriamo un esempio: poniamo che il client abbia richiesto al server un file riservato e poniamo che l'utente si sia identificato e che quindi abbia ricevuto correttamente il file. Se l'utente effettua un'altra richiesta di un file che richiede la stessa identificazione, allora il client (browser) non andrà a richiederne nuovamente di inserire le credenziali ma copierà il *basic-cookie* (che precedentemente si era salvato) nella nuova richiesta. In questo modo il server riconosce che l'utente è sempre lo stesso e risponde inviando il file. I *basic-cookie* (o in generale i cookies) sono informazioni di stato che permettono di creare delle **sessioni**.

I cookies vengono inventati per identificare il fatto che tutte le richieste provengono da uno stesso utente. Nei cookies più moderni non è sempre necessaria l'autenticazione. Basti pensare per esempio ad un sito web di acquisti in cui possiamo aggiungere oggetti al carrello senza esserci autenticati: l'oggetto non viene aggiunto ad un carrello di un utente qualsiasi ma viene aggiunto correttamente al nostro e possiamo effettuare l'acquisto/aggiungere altri articoli. In questi casi è il server che invia un identificatore al client, il quale se lo salva e lo aggiunge ad ogni nuova richiesta tra gli header. In questo modo il server riesce a riconoscere quale utente (anonimo) sta facendo le richieste e il carrello degli acquisti viene individuato correttamente.

I server hanno la possibilità di scambiarsi i cookies e in questo modo è possibile mettere in pratica la profilazione degli utenti (non ha a che vedere con l'identità). La profilazione viene per esempio per la pubblicità mirata.

Tornando al Basic-cookie, abbiamo detto in precedenza che la codifica base64 non è una codifica segreta. Di fatto, utilizzando il *Basic Authentication Scheme*, stiamo inviando il nome utente e la password in chiaro. La sicurezza e la riservatezza vengono garantite dalla presenza di un canale sicuro. La creazione del canale viene gestita da un layer aggiuntivo (facente parte del session layer) chiamato **SSL/TLS** (*Secure Socket Layer/Transport Layer Security*) attraverso l'utilizzo (per esempio) della libreria *OpenSSL* (interfaccia con il layer

SSL/TLS). Dunque non viene utilizzata, come abbiamo visto nei capitoli precedenti, la system call `connect()` (interfaccia al transport layer). Tutti i dati che fluiscono attraverso il canale sono criptati e di conseguenza non è possibile risalire al significato dei singoli byte. Uno degli algoritmi più utilizzati per criptare i dati è l'**algoritmo di Diffie-Hellman**: permette la creazione di un segreto usa e getta tra due peer che generano le proprie chiavi indipendentemente.

11 Common Gateway Interface

Siamo interessati a capire come far comunicare un client con un processo in backend che gira sul server.

11.1 URI

Abbiamo sempre trattato l'URI come il nome di un file, tuttavia gli URL a cui siamo più abituati non assomigliano per niente al un nome di un file. Nella sezione 3.2.1 della RFC 1945 troviamo la definizione di URI:

```
1  URIs in HTTP can be represented in absolute form or relative to some
2  known base URI [9], depending upon the context of their use. The two
3  forms are differentiated by the fact that absolute URIs always begin
4  with a scheme name followed by a colon
5
6  URI          = ( absoluteURI | relativeURI ) [ "#" fragment ]
7
8  absoluteURI  = scheme ":" *( uchar | reserved )
9
10 relativeURI  = net_path | abs_path | rel_path
11
12 net_path     = "//" net_loc [ abs_path ]
13 abs_path     = "/" rel_path
14 rel_path     = [ path ] [ ";" params ] [ "?" query ]
15
16 path         = fsegment *( "/" segment )
17 fsegment     = 1*pchar
18 segment     = *pchar
19
20 params       = param *( ";" param )
21 param        = *( pchar | "/" )
22
23 scheme       = 1*( ALPHA | DIGIT | "+" | "-" | "." )
24 net_loc      = *( pchar | ";" | "?" )
25 query        = *( uchar | reserved )
26 fragment     = *( uchar | reserved )
27
28 pchar        = uchar | ":" | "@" | "&" | "=" | "+"
29 uchar        = unreserved | escape
30 unreserved   = ALPHA | DIGIT | safe | extra | national
31
32 escape       = "%" HEX HEX
33 reserved     = ";" | "/" | "?" | ":" | "@" | "&" | "=" | "+"
34 extra        = "!" | "*" | "'" | "(" | ")" | ","
35 safe         = "$" | "-" | "_" | "."
36 unsafe       = CTL | SP | "<" | ">" | "#" | "%" | "<" | ">"
37 national     = <any OCTET excluding ALPHA, DIGIT,
38               reserved, extra, safe, and unsafe>
```

Possiamo notare che un URI ha una definizione molto generale, non segue uno schema ben preciso. Nella sezione 3.2.2 della stessa RFC troviamo anche la definizione di URL:

```
1  The "http" scheme is used to locate network resources via the HTTP
2  protocol. This section defines the scheme-specific syntax and
3  semantics for http URLs.
```

```

4      http_URL      = "http:" "/" host [ ":" port ] [ abs_path ]
5
6      host          = <A legal Internet host domain name
7                      or IP address (in dotted-decimal form),
8                      as defined by Section 2.1 of RFC 1123>
9
10     port          = *DIGIT
11
12
13     If the port is empty or not given, port 80 is assumed. The semantics
14     are that the identified resource is located at the server listening
15     for TCP connections on that port of that host, and the Request-URI
16     for the resource is abs_path. If the abs_path is not present in the
17     URL, it must be given as "/" when used as a Request-URI (Section
18     5.1.2).
19
20     Note: Although the HTTP protocol is independent of the transport
21     layer protocol, the http URL only identifies resources by their
22     TCP location, and thus non-TCP resources must be identified by
23     some other URI scheme.
24
25     The canonical form for "http" URLs is obtained by converting any
26     UPALPHA characters in host to their LOALPHA equivalent (hostnames are
27     case-insensitive), eliding the [ ":" port ] if the port is 80, and
28     replacing an empty abs_path with "/".

```

Notiamo che l'URL rispetta la sintassi di un URI, infatti non è altro che una specializzazione di un URI: si tratta di un *absoluteURI*⁵. Tuttavia non è l'unico URI, infatti su www.iana.org è possibile trovare un elenco completo di tutti gli URI registrati: troviamo sia *http* che *https*. Altri esempi di URI sono: *file* che permette di visualizzare sul browser file presenti su disco; *geo* che permette di visualizzare locazioni geografiche (visualizzabili per esempio con google maps); *ftp* che sta per *File Transfer Protocol*, permette di trasferire dei file, viene richiesto di aprire un'applicazione esterna che offre tale servizio. Sono tre esempi che non centrano direttamente con il web, infatti molti degli URI si collegano ad applicazioni esterne, altri invece funzionano direttamente su browser. L'utilità sta nel fatto che in questo modo possiamo collegare ed interagire con altre applicazioni direttamente da una pagina web.

11.2 Query

I *relativeURI* (sempre definiti nella RFC 1945 sezione 3.2.1) invece riguardano le risorse di rete. La *query* (in *rel_path*) che viene definita nella grammatica permette di inviare dei dati al server: si tratta del primo meccanismo di web dinamico che è stato implementato.

```

1      <!DOCTYPE html>
2      <html>
3          <body>
4              <!-- Nel campo action si trova l'URI -->
5              <form action="/cgi/cgiexe" method="GET" >
6                  <input type="text" id="box" name="Casella1">
7                  <input type="text" id="box" name="Casella2">
8                  <input type="submit">
9              </form>

```

⁵Un URL è un *absoluteURI* se consideriamo che questo punta ad un file. Sarebbe il tipo di URL che abbiamo trattato fino ad ora.

```

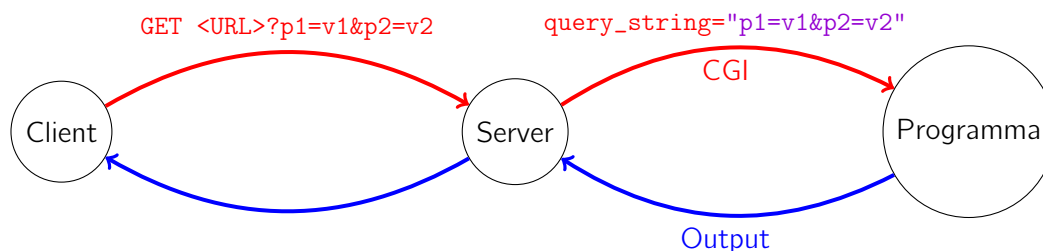
10     </body>
11 </html>

```

Il codice riportato permette di creare un form web che genera un URL basandosi sui valori specificati per i campi *Casella1* e *Casella2*. Salvando il codice html in un file, aprendolo con un browser e inserendo nei rispettivi campi *val1* e *val2*, si ottiene il seguente URI:

```
file:///cgi/cgiexe?Casella1=val1&Casella2=val2
```

I parametri vengono inseriti direttamente nella GET e questi vengono ricevuti dal server e passati come argomenti ad un programma che gira in backend. Questo interfacciamento tra web server e applicazione esterna (o programma) prende il nome di **Common Gateway Interface (CGI)**.



Riassumendo:

1. Il client invia la richiesta utilizzando il metodo *GET* e aggiungendo all'URL dei parametri e i loro rispettivi valori. Nel nostro esempio abbiamo `GET <URL>?p1=v1&p2=v2`.
2. Il server web estrae la query string `query_string = "p1=v1&p2=v2"` e la passa come argomento al programma eseguibile che gira in backend. Più precisamente la `query_string` viene passata come variabile d'ambiente.
3. Se tutto va a buon fine, il server invia al client la status line e gli header. L'entity body verrà inviato dal programma eseguibile in backend.
4. Il programma effettua il parsing della `query_string`, processa la richiesta e produce l'output. L'output prodotto rappresenta l'entity body della response che viene inviato al client, dal programma eseguibile, mediante il server.

Il nome della variabile d'ambiente `query_string` passata al programma in backend, non è scelto arbitrariamente. Nella RFC 3875 (dedicata alla definizione della *Common Gateway Interface*) nella sezione 4.1 si trovano i nomi standard delle variabili d'ambiente.

```

1 [4.1 Request Meta-Variables]
2
3 Meta-variables contain data about the request passed from the server
4 to the script, and are accessed by the script in a system-defined
5 manner. Meta-variables are identified by case-insensitive names;
6 there cannot be two different variables whose names differ in case
7 only. Here they are shown using a canonical representation of
8 capitals plus underscore ("_"). A particular system can define a
9 different representation.
10
11 meta-variable-name = "AUTH_TYPE" | "CONTENT_LENGTH" |
12                      "CONTENT_TYPE" | "GATEWAY_INTERFACE" |
13                      "PATH_INFO" | "PATH_TRANSLATED" |

```

```

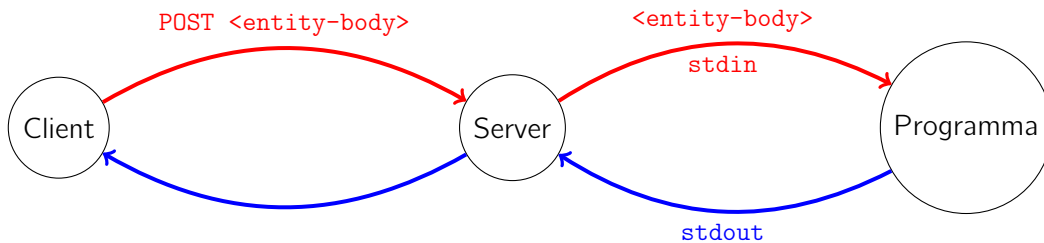
14         "QUERY_STRING" | "REMOTE_ADDR" |
15         "REMOTE_HOST" | "REMOTE_IDENT" |
16         "REMOTE_USER" | "REQUEST_METHOD" |
17         "SCRIPT_NAME" | "SERVER_NAME" |
18         "SERVER_PORT" | "SERVER_PROTOCOL" |
19         "SERVER_SOFTWARE" | scheme |
20         protocol-var-name | extension-var-name
21 protocol-var-name = ( protocol | scheme ) "_" var-name
22 scheme            = alpha *( alpha | digit | "+" | "-" | "." )
23 var-name          = token
24 extension-var-name = token

```

Il meccanismo che abbiamo appena descritto prende il nome di **Request Meta-Variables** nella RFC 3875.

11.3 Il metodo POST

Il metodo *POST* (introdotto in HTTP 1.1) è il successore del metodo *GET* utilizzato con la *query* che abbiamo analizzato nella sottosezione precedente.



Descriviamo i vari passi:

1. Il client invia al web server una richiesta (chiamata **Request Message-Body** nella RFC 3875 sezione 4.2 riportata sotto) in cui specifica nell'entity body i parametri e i valori assegnati da passare al programma in backend. Tra gli header dell'entity body deve essere necessariamente presente il `content_length` e il `content_type`: questi verranno passati come variabili d'ambiente al programma⁶.
2. Il server riceve la richiesta e inoltra l'entity body in input al programma in backend.
3. Il programma effettua il parsing delle variabili d'ambiente, processa la richiesta e invia l'output (response) al client mediante il server.

```

1 [4.2. Request Message-Body]
2
3 Request data is accessed by the script in a system-defined method;
4 unless defined otherwise, this will be by reading the 'standard
5 input' file descriptor or file handle.
6
7 Request-Data = [ request-body ] [ extension-data ]
8 request-body = <CONTENT_LENGTH>OCTET
9 extension-data = *OCTET
10
11 A request-body is supplied with the request if the CONTENT_LENGTH is
12 not NULL. The server MUST make at least that many bytes available

```

⁶In questo caso non serve specificare nessuna `query_string`.

```
13 for the script to read. The server MAY signal an end-of-file
14 condition after CONTENT_LENGTH bytes have been read or it MAY supply
15 extension data. Therefore, the script MUST NOT attempt to read more
16 than CONTENT_LENGTH bytes, even if more data is available. However,
17 it is not obliged to read any of the data.
18
19 For non-parsed header (NPH) scripts (section 5), the server SHOULD
20 attempt to ensure that the data supplied to the script is precisely
21 as supplied by the client and is unaltered by the server.
```

Nella sezione 6.1 della RFC 3875 troviamo informazioni riguardo l'output che deve fornire il programma in backend.

```
1 [6.1. Response Handling]
2
3 A script MUST always provide a non-empty response, and so there is a
4 system-defined method for it to send this data back to the server.
5 Unless defined otherwise, this will be via the 'standard output' file
6 descriptor.
7
8 The script MUST check the REQUEST_METHOD variable when processing the
9 request and preparing its response.
10
11 The server MAY implement a timeout period within which data must be
12 received from the script. If a server implementation defines such a
13 timeout and receives no data from a script within the timeout period,
14 the server MAY terminate the script process.
```

11.4 ws_cgi.c

```
1 #include <stdio.h>
2 #include <sys/socket.h>
3 #include <errno.h>
4 #include <arpa/inet.h>
5 #include <unistd.h>
6 #include <string.h>
7 #include <stdlib.h>
8 #include <sys/wait.h>
9
10 /* Struttura di un header:
11     Nome | Valore
12 */
13 struct headers {
14     char* name;
15     char* value;
16 } h[100];
17
18 unsigned char envbuf[1000];
19 int env_c, env_i;
20 char* env[100];
21
22 void print_client_ip(in_addr_t addr) {
23     unsigned char* ptr = (unsigned char*) &addr;
24     for (int i = 0; i < 4; i++) {
```

```

25     printf("%u", ptr[i]);
26     if (i != 3) printf(".");
27 }
28 }
29
30 void add_env(const char* env_key, const char* env_value){
31     sprintf(envbuf+env_c, "%s=%s", env_key, env_value);
32     env[env_i++] = envbuf+env_c;
33     env_c += (strlen(env_value)+strlen(env_key)+2);
34     env[env_i]=NULL;
35 }
36
37 int main(void) {
38     char request[3001];
39     char response[3001];
40
41     int s = socket(AF_INET, SOCK_STREAM, 0);
42     if (s == -1) {
43         perror("Socket fails");
44         return 1;
45     }
46
47     struct sockaddr_in srvaddr;
48     srvaddr.sin_family = AF_INET;
49     srvaddr.sin_port = htons(9010);
50     srvaddr.sin_addr.s_addr = 0; // 0 => indirizzo della macchina.
51
52
53     /* Evita errori generati dalla bind su indirizzi utilizzati
54        recentemente */
55     int yes = 1;
56     int t = setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
57     if (t == -1) {
58         perror("Setsockopt fails");
59         return 1;
60     }
61
62     // ### Bind ###
63     /* Rendo passivo il socket sfruttando la bind */
64     t = bind(s, (struct sockaddr*) &srvaddr, sizeof(struct sockaddr_in));
65     if (t == -1) {
66         perror("Bind fails");
67         return 1;
68     }
69
70     // ### Listen ###
71     /* Rendo il socket passivo e fisso la dimensione della coda delle richieste
72        pendenti a 5. */
73     t = listen(s, 5);
74     if (t == -1) {
75         perror("Listen fails");
76         return 1;
77     }
78
79     // ### Accept ###
80     /* Accetto le richieste. */

```

```
81 struct sockaddr_in remote;
82 int len = sizeof(struct sockaddr);
83
84 int s2 = 0; // File descriptor del socket tornato da accept.
85 while (1) {
86     close(s2);
87     s2 = accept(s, (struct sockaddr*) &remote, &len);
88
89     /* Faccio gestire ai processi figli le richieste */
90     if (fork()) continue;
91
92     if (s2 == -1) {
93         perror("Accept fails");
94         exit(1);
95     }
96
97     // Stampo informazioni client.
98     printf("\nClient port = %u, addr = ", htons(remote.sin_port));
99     print_client_ip(remote.sin_addr.s_addr);
100    printf("\n\n");
101
102    // ### Parsing Header ###
103    char hbuf[10000]; // Buffer header.
104
105    h[0].name = "Request Line";
106    h[0].value = hbuf;
107
108    int i, j;
109    for (i = 0, j = 0; read(s2, hbuf + i, 1); i++) {
110
111        // Setto il puntatore al Valore.
112        if (hbuf[i] == ':' && h[j].value == NULL) {
113            h[j].value = &hbuf[i+1]; // Il +1 extra e' per evitare lo
114                spazio.
115            hbuf[i] = 0; // Terminatore di stringa.
116        }
117
118        // Setto il puntatore al Nome o termino.
119        else if (i > 0 && hbuf[i] == '\n' && hbuf[i-1] == '\r') {
120            hbuf[i-1] = 0; // Terminatore di stringa.
121
122            /* Controllo se l'header e' terminato.
123               L'header termina quando ho un doppio 'CRLF'. */
124            if (h[j].name[0] == 0) {
125                break;
126            }
127            // Setto il puntatore al Nome.
128            h[++j].name = &hbuf[i+1];
129        }
130    }
131
132    // Stampa header.
133    int content_length = 0;
134    for (i = 0; i < j; i++) {
135        printf("%s:%s\n", h[i].name, h[i].value);
136        if (!strcmp(h[i].name, "Content-Length")) {
```

```

136         content_length = atoi(h[i].value);
137     }
138 }
139 printf("\n");
140
141 // ### Parsing Request-Line ###
142 // Request-Line = Method SP Request-URI SP HTTP-Version CRLF
143 char *method, *filename, *http_version;
144 method = h[0].value;
145 for (i = 0; h[0].value[i] != ' '; i++) ;
146 h[0].value[i] = 0;
147 i += 1;
148
149 filename = h[0].value + i;
150 for ( ; h[0].value[i] != ' '; i++) ;
151 h[0].value[i] = 0;
152 i += 1;
153
154 http_version = h[0].value + i; // Il terminatore e' stato inserito
155                                // (al posto di '\r') nel parser
156                                // dell'header.
157
158 // ### Response ###
159 /* Gestione CGI. */
160 if (!strcmp("/cgi/", filename, 5)) {
161     filename += 5;
162     if (!strcmp("GET", method)) {
163         for (i = 0; filename[i] && filename[i] != '?'; i++) ;
164         if (filename[i] == '?') {
165             filename[i] = 0;
166             add_env("QUERY_STRING", filename + i + 1);
167         }
168         add_env("CONTENT_LENGTH", "0");
169     } else if (!strcmp("POST", method)) {
170         char tmp[10];
171         sprintf(tmp, "%d", content_length);
172         add_env("CONTENT_LENGTH", tmp);
173     } else {
174         sprintf(response, "HTTP/1.1 501 Not Implemented\r\n\r\n");
175         write(s2, response, strlen(response));
176         close(s2);
177         continue;
178     }
179
180     /* Full-Response. */
181     FILE* fin = fopen(filename, "rt");
182     if (fin == NULL) {
183         sprintf(response, "HTTP/1.1 404 NOT
184             FOUND\r\nConnection:close\r\n\r\n<html><h1>404 File %s
185             found</h1></html>", filename);
186         write(s2, response, strlen(response));
187         close(s2);
188         exit(1);
189     }
190 }

```



```

189     sprintf(response, "HTTP/1.1 200 OK\r\nConnection: close\r\nDate: Sat,
190         27 Apr 2024 10:27:00 GMT\r\n\r\n");
191     write(s2, response, strlen(response));
192
193     for(i = 0; env[i]; i++) {
194         printf("environment: %s\n", env[i]);
195     }
196
197     char fullname[100];
198     sprintf(fullname,
199         "/home/luca/Desktop/c_cpp/reti_dei_calcolatori/webserver/ws_cgi/%s",
200         filename);
201
202     char* myargv[3];
203     myargv[0] = fullname;
204     myargv[1] = NULL;
205
206     int pid = 0;
207     if (!(pid = fork())) {
208         dup2(s2, 1); // Connetto stdout al socket connesso al client.
209         dup2(s2, 0); // Connetto stdin al socket connesso al client.
210         if (execve(fullname, myargv, env) == -1) {
211             perror("execve");
212             exit(0);
213         }
214     }
215     waitpid(pid, NULL, 0);
216     printf("Il processo figlio e' terminato.\n");
217
218     fclose(fin);
219 } else {
220     /* Full-Response: gestione file richiesto. */
221     FILE* fin = fopen(filename + 1, "rt"); // Il '+1' per rimuovere
222                                             // il path relativo.
223     if (fin == NULL) {
224         sprintf(response, "HTTP/1.1 404 NOT
225             FOUND\r\nConnection:close\r\n\r\n<html><h1>404 File %s
226             found</h1></html>", filename);
227         write(s2, response, strlen(response));
228         close(s2);
229         exit(1);
230     }
231
232     sprintf(response, "HTTP/1.1 200 OK\r\nConnection:
233         close\r\nContent-Type: text/html\r\nDate: Sat, 27 Apr 2024
234         10:27:00 GMT\r\n\r\n");
235     write(s2, response, strlen(response));
236
237     char entity[1000];
238     while (!feof(fin)) {
239         fread(entity, 1, 1000, fin);
240         write(s2, entity, 1000);
241     }
242 }

```

```

238     exit(0); // Per far terminare il processo figlio.
239 }
240
241 return 0;
242 }

```

```

1 // cgiexe.c
2
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>
6 #include <unistd.h>
7
8 int main(int argc, char * argv[], char * env[]){
9     int i,j,t;
10    int length=0;
11    char line[500],*key,*value,*buffer;
12
13    printf("Processo: ho questo environment:\n");
14    for(i=0;env[i];i++){
15        key = env[i];
16        for(j=0;env[i][j]!='=';j++);
17        env[i][j]=0;
18        value = env[i]+j+1;
19        if(!strcmp(key,"CONTENT_LENGTH"))
20            length = atoi(value);
21    }
22    fflush(stdout);
23
24    buffer=malloc(length);
25
26    for(i=0;i<length && (t=read(0,buffer+i,length-i));i+=t) ;
27    printf("Body received\n");
28
29    for(i=0;i<length && (t=write(1,buffer+i,length-i));i+=t) ;
30
31    for(i=0;env[i];i++)
32        printf("%s---> %s\n",env[i],env[i]+strlen(env[i])+1);
33
34    return 0;
35 }

```

12 Domain Name System

Il principale ruolo che ricopre un DNS può essere descritto con una semplice frase: permettono di mappare nomi in indirizzi ip. Quando inseriamo nella barra di ricerca di un browser la stringa `www.google.com`, questa viene **risolta** e viene trovato l'indirizzo ip corrispondente. Prima dei DNS, per poter risolvere gli indirizzi, si trovava nei sistemi il file `/etc/hosts` in cui veniva memorizzata una tabella composta dal nome e l'indirizzo ip associato al server. Oltre a `/etc/hosts` che era memorizzato⁷ in locale, veniva utilizzato un altro file, molto più grande, memorizzato nei *Network Information Center* (NIC). Gli amministratori di rete riempivano le tabelle dei NIC e i client potevano risolvere gli indirizzi andando a richiedere le tabelle. Si è deciso di trovare una soluzione alternativa ai NIC quando i nodi della rete iniziavano a crescere in maniera vertiginosa e di conseguenza anche le dimensioni delle tabelle, inoltre c'era da considerare che tali tabelle dovevano anche essere trasmesse a tutti i nodi della rete e ciò avrebbe causato degli ingombri notevoli.

Per sopperire a queste problematiche vengono introdotti, nella RFC 1034 dell'anno 1987, i DNS. Si tratta di un progetto talmente ben fatto che si utilizza tutt'oggi tale e quale a come è stato proposto per la prima volta.

12.1 Gli obiettivi del DNS

Gli obiettivi posti con l'introduzione dei DNS erano i seguenti:

1. sistema auto-consistente: non deve appoggiarsi a nessun'altra sorgente di informazione;
2. sistema distribuito: no informazioni replicate come avveniva per i NIC; deve essere scalabile;
3. supporto multiprotocollo: riguardo ai record supportati, infatti i DNS non supportano solamente gli indirizzi ip ma anche altri protocolli;
4. multiapplicazione: non supportano solamente la ricerca di indirizzi (grazie al fatto che i record hanno un tipo definito);
5. compromesso tra efficienza ed accuratezza.

12.2 La struttura dei nomi e la risoluzione di un nome

I nomi sono strutturati in maniera gerarchica come mostrato in Figura 12. La struttura gerarchica permette di categorizzare gli indirizzi.

Per capire il funzionamento consideriamo un esempio: vogliamo risalire all'indirizzo di `www.dei.unipd.it`⁸. Innanzitutto introduciamo il **resolver**: un software che si occupa di trovare l'indirizzo ip associato al nome che abbiamo specificato. Il resolver può essere impostato sul browser che utilizziamo, una scelta comune è quello di google che si trova all'indirizzo 8.8.8.8. Dunque il resolver riceve in input `www.dei.unipd.it` e il suo primo compito è quello di trovare un **root server**. I root server totali nel mondo⁹ sono 13, il resolver si connette al primo disponibile e chiede informazioni relative al **name server** `.it`. Il root server risponde inviando l'indirizzo associato a `.it` al quale il resolver si collegherà per chiedere l'indirizzo del name server `unipd.it`. Ricevuto l'indirizzo associato a `unipd.it`, il resolver si collega e richiede l'indirizzo del name server `dei.unipd.it` al quale infine chiede l'indirizzo del name server `www.dei.unipd.it` e lo restituisce al client.

⁷Si trova tuttora nei sistemi unix.

⁸Il nome corretto sarebbe `www.dei.unipd.it`, ma il punto finale viene sottointeso.

⁹In realtà i nomi sono 13 (dalla A alla M), i server sono replicati: esistono più macchine che rispondono ad uno stesso nome.

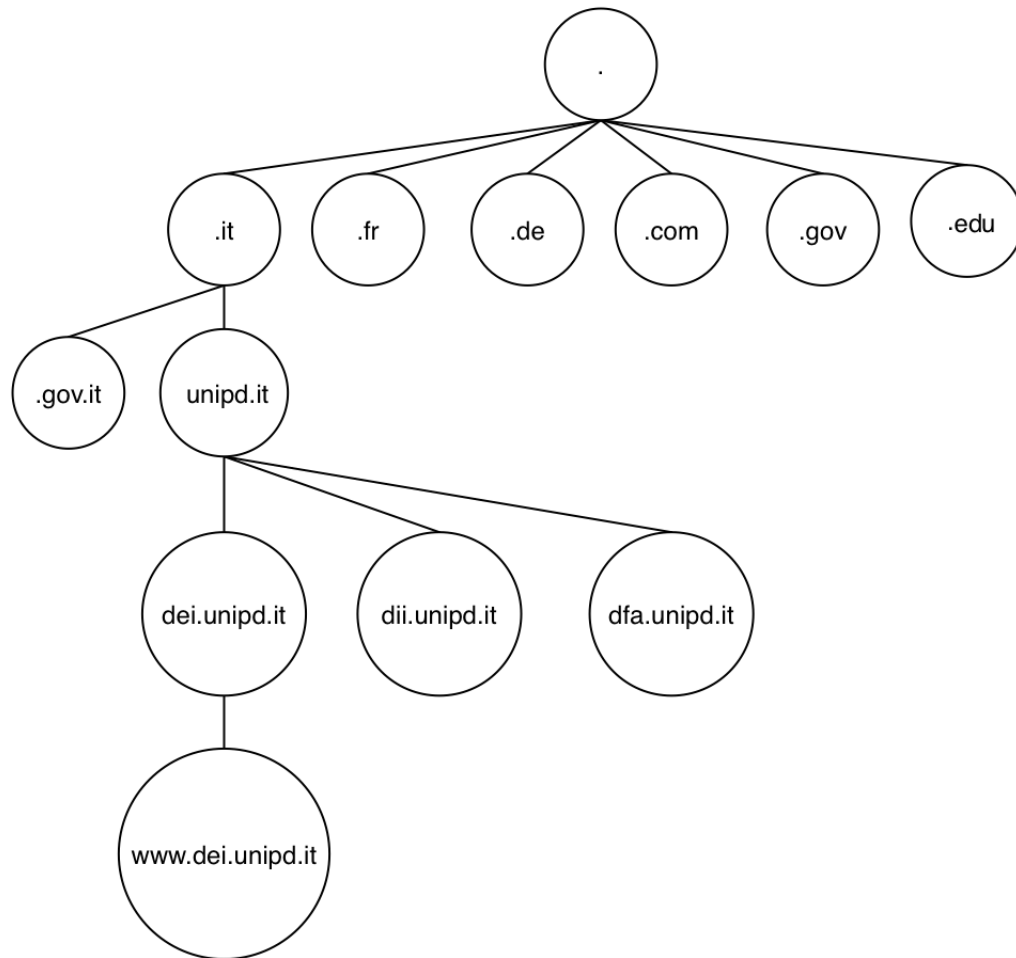


Figura 12: Spazio dei nomi.

Ogni genitore rappresenta un server che contiene i record relativi ai suoi figli. La risoluzione di un nome consiste in un botta e risposta effettuato tra resolver e un nodo genitore. Questo processo va avanti finché non si raggiunge una foglia (oppure non ci si blocca prima causa name server richiesto non trovato).

12.3 Gestione dei cambiamenti

TODO

12.4 Le zone

Precedentemente abbiamo considerato che ogni nodo dell'albero fosse associato ad un server, in realtà non è esattamente così. Si possono infatti creare delle **zone** (sottoalberi) che comprendono più nodi e indicano che un solo server è autoritativo per quei name server. Nell'esempio di Figura 13 viene mostrata una zona che comprende tre nodi. In questo caso il server di `unipd.it` è autoritativo rispetto ad entrambi i nodi che appartengono alla zona. Il vantaggio è che nella risoluzione di un nome si effettuano meno passaggi, infatti da `unipd.it` possiamo risalire direttamente a `www.dei.unipd.it`.

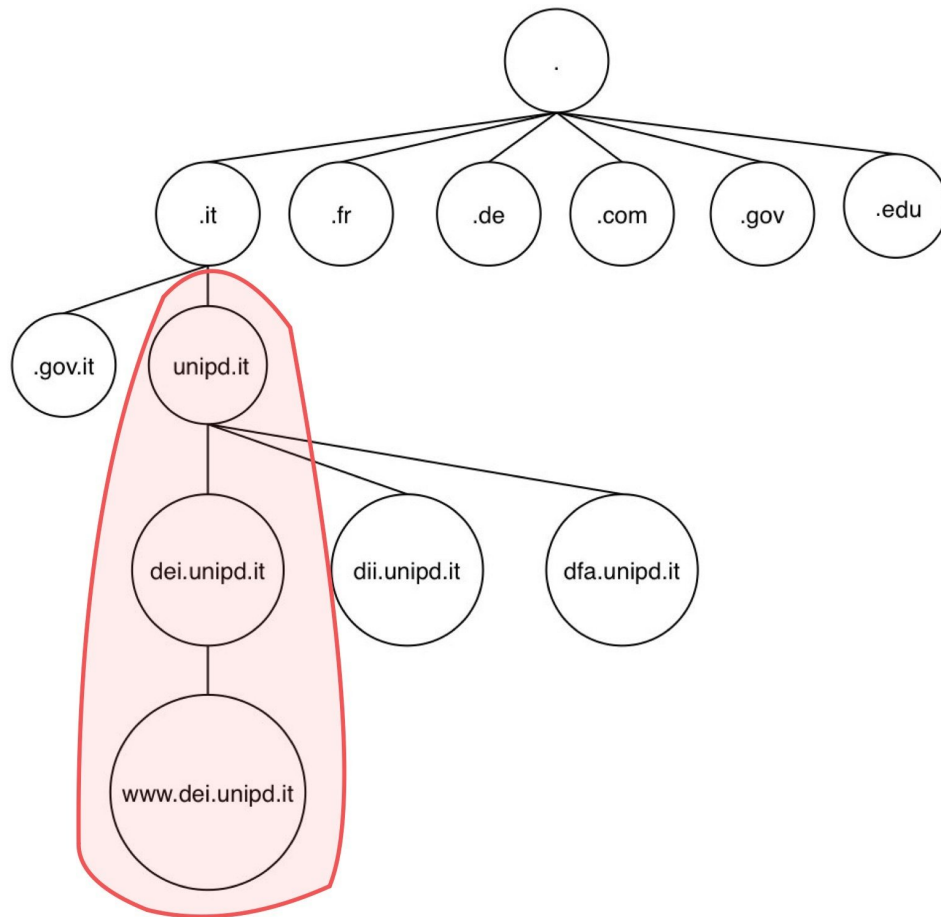


Figura 13: Esempio di zona.

12.5 DNS non-recursive query chain

Analizziamo in pratica come si risolve un nome in maniera non ricorsiva. Il funzionamento della risoluzione non ricorsiva lo abbiamo descritto nella sottosezione 12.2, nella prossima sottosezione vedremo la differenza con il più utilizzato metodo ricorsivo.

Utilizziamo il comando `dig` su linux. Il primo passo è quello di andare a richiedere i name server di root:

```

1 >> dig . +norecurse NS
2 ;; ANSWER SECTION:
3 87203 IN NS e.root-servers.net.
4 87203 IN NS f.root-servers.net.
5 87203 IN NS a.root-servers.net.

```

Notiamo che la risposta non è autoritativa (infatti nei flag non compare aa). Ora possiamo risolvere per esempio `a.root-servers.net` per ricavare l'indirizzo del root server:

```

1 >> dig a.root-servers.net +norecurse A
2 ;; ANSWER SECTION:
3 a.root-servers.net. 259186 IN A 198.41.0.4

```

Possiamo richiedere al root server i name server e gli indirizzi degli altri root server.

```

1 >> dig @198.41.0.4 . +norecurse NS
2 ;; ANSWER SECTION:
3 518400 IN NS e.root-servers.net.
4 518400 IN NS f.root-servers.net.
5 518400 IN NS a.root-servers.net.
6 ;; ADDITIONAL SECTION
7 a.root-servers.net. 518400 IN A 198.41.0.4 // Stesso indirizzo trovato nella
risposta non autoritativa precedente.

```

In questo caso la risposta è autoritativa. Richiedo al root server informazioni sul dominio .it.

```

1 >> dig @198.41.0.4 it. +norecurse NS
2 ;; AUTHORITY SECTION: // I server autoritativi (glue record).
3 it. 172800 IN NS a.dns.it.
4 it. 172800 IN NS m.dns.it.
5 it. 172800 IN NS dns.nic.it.
6 it. 172800 IN NS d.dns.it.
7 it. 172800 IN NS nameserver.cnr.it.
8 it. 172800 IN NS r.dns.it.
9 ;; ADDITIONAL SECTION: // Indirizzi dei server autoritativi.
10 a.dns.it. 172800 IN A 194.0.16.215
11 a.dns.it. 172800 IN AAAA 2001:678:12:0:194:0:16:215
12 nameserver.cnr.it. 172800 IN A 194.119.192.34
13 nameserver.cnr.it. 172800 IN AAAA 2a00:1620:c0:220:194:119:192:34
14 ...

```

Ottingo una risposta non autoritativa (root non è autoritativo su .it). I glue record vengono aggiornati dai server autoritativi, devono fare un push al server che si trova un livello sopra nella struttura ad albero. Tale meccanismo permette la risoluzione del nome. Ora posso interrogare un server autoritativo per ottenere una risposta autoritativa.

```

1 >> dig @194.119.192.34 it. +norecurse NS
2 ;; ANSWER SECTION:
3 it. 3600 IN NS d.dns.it.
4 it. 3600 IN NS a.dns.it.
5 it. 3600 IN NS r.dns.it.
6 it. 3600 IN NS dns.nic.it.
7 it. 3600 IN NS nameserver.cnr.it.
8 it. 3600 IN NS m.dns.it.

```

La risposta ottenuta è pressoché identica a quella precedente ma in questo caso è autoritativa (non sono glue record). Ora posso chiedere informazioni riguardo unipd.it.

```

1 >> dig @194.119.192.34 unipd.it. +norecurse NS
2 ;; AUTHORITY SECTION:
3 unipd.it. 3600 IN NS mail.dei.unipd.it.
4 unipd.it. 3600 IN NS ns1.garr.net.
5 unipd.it. 3600 IN NS ns1.unipd.it.
6 unipd.it. 3600 IN NS giotto.unipd.it.
7 ;; ADDITIONAL SECTION:
8 giotto.unipd.it. 3600 IN A 147.162.100.180
9 mail.dei.unipd.it. 3600 IN A 147.162.2.100
10 ns1.unipd.it. 3600 IN A 147.162.1.42

```

Procedo chiedendo di dei.unipd.it.

```
1 >> dig @147.162.100.180 dei.unipd.it. +norecurse NS
2 ;; AUTHORITY SECTION:
3 dei.unipd.it. 86400 IN NS mail.dei.unipd.it.
4 dei.unipd.it. 86400 IN NS ns1.dii.unipd.it.
5 dei.unipd.it. 86400 IN NS ns1.unipd.it.
6 ;; ADDITIONAL SECTION:
7 mail.dei.unipd.it. 86400 IN A 147.162.2.100
8 ns1.dii.unipd.it. 86400 IN A 147.162.24.1
```

Ricaviamo la risposta autoritativa per verificare la correttezza degli ip.

```
1 >> dig @147.162.24.1 dei.unipd.it. +norecurse NS
2 ;; ANSWER SECTION:
3 dei.unipd.it. 172800 IN NS dns.unipd.it.
4 dei.unipd.it. 172800 IN NS ns1.dii.unipd.it.
5 dei.unipd.it. 172800 IN NS mail.dei.unipd.it.
6 ;; ADDITIONAL SECTION:
7 ns1.dii.unipd.it. 3600 IN A 147.162.24.1
8 mail.dei.unipd.it. 172800 IN A 147.162.2.100
```

Infine ricaviamo l'indirizzo di www.dei.unipd.it.

```
1 >> dig @147.162.24.1 www.dei.unipd.it. +norecurse A
2 ;; ANSWER SECTION:
3 www.dei.unipd.it. 172800 IN CNAME www1.dei.unipd.it. // CNAME indica che
4 www.dei.unipd.it e' un alias di www1.dei.unipd.it.
5 www1.dei.unipd.it. 172800 IN A 147.162.2.199
```

Il risultato della query è dunque 147.162.2.199.

12.6 DNS recursive query

La query ricorsiva è molto più efficiente: evita i possibili colli di bottiglia che potrebbero generarsi dalle molteplici richieste effettuate ai nodi interni dell'albero dei DNS. Descriviamo il funzionamento ricorsivo considerando un esempio: poniamo di volere risolvere `www.dei.unipd.it` e poniamo di conoscere l'indirizzo del name server `.it`. Chiediamo al server di `.it` di risolvere `www.dei.unipd.it`; a sua volta questo chiederà al server di `unipd.it` di risolvere l'indirizzo; il quale a sua volta chiederà a `dei.unipd.it` di risolvere l'indirizzo; e così via fino al raggiungimento dell'indirizzo del server desiderato. Infine l'indirizzo verrà restituito all'host dal primo server a cui si è rivolto (`.it` nel nostro esempio).

12.7 DNS caching

Dopo aver eseguito una query ricorsiva, la risposta ottenuta viene solitamente salvata nella cache dell'utente. Anche il server (nell'esempio precedente `.it`) si salva in cache il risultato della richiesta, in questo modo potrà rispondere immediatamente a richieste successive (anche di altri host).

Utilizzando invece la query non ricorsiva non viene salvato il risultato nella cache. Il vantaggio è che questa tipologia di query fornisce un dato sicuro, mentre la variante ricorsiva richiede che la cache sia opportunamente aggiornata.

L'aggiornamento dei dati nella cache avviene dopo un certo tempo prestabilito. Il valore dopo il quale il dato in cache risulta obsoleto viene fornito effettuando una richiesta ad

un server autoritativo: restituisce un timer (in secondi) che, una volta terminato, indica la necessità di aggiornare il dato in cache.

13 Il proxy

13.1 Il proxy in HTTP

Il proxy è un software che lavora nei livelli applicativi, funge da intermediario. Il suo funzionamento è molto semplice:

1. Il client effettua la richiesta *GET http://example.com/x/y/z* al server proxy.
2. Il proxy non ha le informazioni necessarie per soddisfarla, quindi effettua la richiesta *GET /x/y/z* attraverso il proprio client, dopo essersi connesso a *example.com*.
3. Infine il proxy riceve la risposta dal server e la gira al client.

L'URI fornito dal client è composto da due parti fondamentali:

- L'**hostname** *http://example.com*: per far sì che il client proxy possa collegarsi con il server di *example.com*. L'ip del server si ottiene mediante una DNS query.
- Il **percorso** */x/y/z*.

Il client deve essere configurato per lavorare in modalità proxy e deve conoscere l'indirizzo ip di un server proxy.

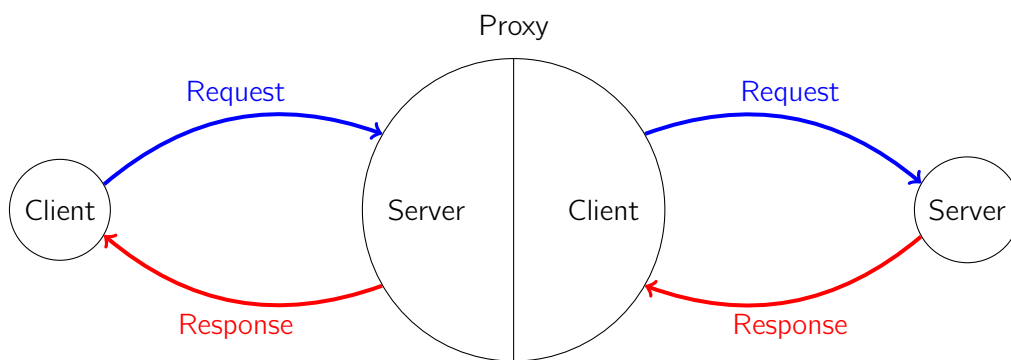


Figura 14: Funzionamento di un proxy.

In Figura 15, gli host si interfacciano alla rete attraverso il proxy: non hanno associato un indirizzo ip pubblico ma solamente privato¹⁰; sono di fatto isolati da internet, protetti dal proxy che ricopre il ruolo di un firewall. Per questa sua funzionalità, il proxy viene anche chiamato **gateway applicativo**.

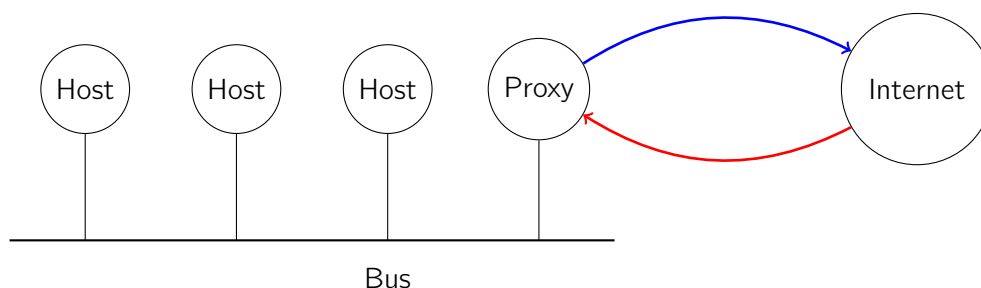


Figura 15: Host connessi da un bus comune

¹⁰Indirizzo relativo alla rete privata

Altre funzionalità offerte dal proxy:

- servizio di caching molto efficiente;
- filtraggio delle richieste: su URL e contenuti delle pagine scaricate;
- possibilità di salvarsi un file di log dei contenuti richiesti.

Web Application Firewall Il proxy, per come lo abbiamo definito, viene utilizzato a servizio del client. Esiste anche una variante lato server che prende il nome di **Web Application Firewall (WAF)**.

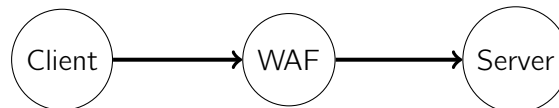


Figura 16: Funzionamento di un WAF.

Il client pensa di connettersi al server, in realtà si connette al WAF. Il WAF analizza la richiesta del client per verificare la presenza di attacchi: funge da filtro sulle richieste. Se tutto va a buon fine, la richiesta raggiunge il server.

13.2 Il proxy in HTTPS

Come abbiamo visto nella precedente sottosezione, il proxy lavora con i dati in chiaro: non sono crittografati, vede le richieste degli host e vede i contenuti delle risposte fornite dai server. In HTTPS viene garantita la privacy con la creazione di un **tunnel** in cui i dati sono criptati. In questo modo il proxy non vede le richieste e le risposte in chiaro ma semplicemente dei byte; il suo unico compito è quello di inoltrarli verso il client o verso il server. L'instaurazione del canale sicuro avviene come segue:

1. Il client effettua la richiesta *CONNECT server:port* dove *CONNECT* è un metodo di HTTP. Questa richiesta viene effettuata in chiaro.
2. Il proxy riceve la richiesta del client e si occupa di instaurare una connessione sicura con il server specificato dal client.
3. Se tutto va a buon fine, il proxy risponde al client con *HTTP/1.1 200 Established*.
4. Da questo momento in poi il proxy non vede più i dati in chiaro ma solamente byte.

13.3 proxy.c

```

1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <errno.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <string.h>
7  #include <stdlib.h>
8  #include <netdb.h>
9  #include <signal.h>
10
11 /* Struttura di un header:
12     Nome | Valore
  
```

```
13 */
14 struct headers {
15     char* name;
16     char* value;
17 } h[100];
18
19 int main(void) {
20     char request[3001];
21     char response[3001];
22
23     int s = socket(AF_INET, SOCK_STREAM, 0);
24     if (s == -1) {
25         perror("Socket fails");
26         return 1;
27     }
28
29     struct sockaddr_in srvaddr;
30     srvaddr.sin_family = AF_INET;
31     srvaddr.sin_port = htons(9010);
32     srvaddr.sin_addr.s_addr = 0; // 0 => indirizzo della macchina.
33
34     /* Evita errori generati dalla bind su indirizzi utilizzati
35        recentemente */
36     int yes = 1;
37     int t = setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
38     if (t == -1) {
39         perror("Setsockopt fails");
40         return 1;
41     }
42 }
43
44 // ### Bind ###
45 /* Rendo passivo il socket sfruttando la bind */
46 t = bind(s, (struct sockaddr*) &srvaddr, sizeof(struct sockaddr_in));
47 if (t == -1) {
48     perror("Bind fails");
49     return 1;
50 }
51
52 // ### Listen ###
53 /* Rendo il socket passivo e fisso la dimensione della coda delle richieste
54    pendenti a 5. */
55 t = listen(s, 5);
56 if (t == -1) {
57     perror("Listen fails");
58     return 1;
59 }
60
61 // ### Accept ###
62 /* Accetto le richieste. */
63 struct sockaddr_in remote;
64 int len = sizeof(struct sockaddr);
65
66 int s2; // File descriptor del socket tornato da accept.
67 while (1) {
68     close(s2);
```

```

69     s2 = accept(s, (struct sockaddr*) &remote, &len);
70
71     /* Faccio gestire ai processi figli le richieste */
72     if (fork()) continue;
73
74     if (s2 == -1) {
75         perror("Accept fails");
76         exit(1);
77     }
78
79     // Stampo informazioni client.
80     printf("\nClient port = %u, addr = %X\n\n", htons(remote.sin_port),
81           htons(remote.sin_addr.s_addr));
82
83     // ### Parsing Header ###
84     char hbuf[10000]; // Buffer header.
85
86     h[0].name = "Request Line";
87     h[0].value = hbuf;
88
89     int i, j;
90     for (i = 0, j = 0; read(s2, hbuf + i, 1); i++){
91
92         // Setto il puntatore al Valore.
93         if (hbuf[i] == ':' && h[j].value == NULL) {
94             h[j].value = &hbuf[i+1];
95             hbuf[i] = 0; // Terminatore di stringa.
96         }
97
98         // Setto il puntatore al Nome o termino.
99         else if (i > 0 && hbuf[i] == '\n' && hbuf[i-1] == '\r') {
100             hbuf[i-1] = 0; // Terminatore di stringa.
101
102             /* Controllo se l'header e' terminato.
103              L'header termina quando ho un doppio 'CRLF'. */
104             if (h[j].name[0] == 0) {
105                 break;
106             }
107             // Setto il puntatore al Nome.
108             h[++j].name = &hbuf[i+1];
109         }
110     }
111
112     // Stampa header.
113     for (i = 0; i < j; i++) {
114         printf("%s:%s\n", h[i].name, h[i].value);
115     }
116     printf("\n");
117
118     // ### Parsing Request-Line ###
119     // Request-Line = Method SP Request-URI SP HTTP-Version CRLF
120     char *method, *url, *http_version;
121     method = h[0].value;
122     for (i = 0; h[0].value[i] != ' '; i++) ;
123     h[0].value[i] = 0;
124     i += 1;

```

```

124
125     url = h[0].value + i;
126     for ( ; h[0].value[i] != ' '; i++) ;
127     h[0].value[i] = 0;
128     i += 1;
129
130     http_version = h[0].value + i; // Il terminatore e' stato inserito
131                                     // (al posto di '\r') nel parser
132                                     // dell'header.
133
134 // ### Response ###
135     if (!strcmp(method, "GET")) {
136         char* scheme = url;
137         printf("url = %s\n", url);
138
139         // Parsing di scheme, hostame e filename.
140         // Riferimento: GET http://www.example.com/file/file, dove
141         //     scheme = www.example.com
142         //     hostname = www.example.com
143         //     filename = /file/file
144         for (i = 0; url[i] != ':' && url[i]; i++) ;
145         if (url[i] == ':') {
146             url[i++] = 0; // Terminatore di stringa.
147         } else {
148             printf("Parse errore, expected ':'\n");
149             exit(1);
150         }
151         if (url[i] != '/' || url[i+1] != '/') {
152             printf("Parse errore, expected '/'\n");
153             exit(1);
154         }
155         i = i + 2; // Salto i '/'
156         char* hostname = url + i;
157         for ( ; url[i] != '/' && url[i]; i++) ;
158         if (url[i] == '/') {
159             url[i++] = 0; // Terminatore di stringa.
160         } else {
161             printf("Parse errore, expected '/'\n");
162             exit(1);
163         }
164         char* filename = url + i;
165         printf("Schema: %s, hostname: %s, filename: %s\n", scheme, hostname,
166             filename);
167         // Fine parsing.
168
169         // Risolvo il nome.
170         struct hostent* he = gethostbyname(hostname);
171         printf("ip server: %d.%d.%d.%d\n", (unsigned char) he->h_addr[0],
172             (unsigned char) he->h_addr[1], (unsigned char) he->h_addr[2],
173             (unsigned char) he->h_addr[3]);
174
175         // Creazione client del proxy.
176         int s3 = socket(AF_INET, SOCK_STREAM, 0);
177         if (s3 == -1) {
178             perror("Socket fails");
179             exit(1);
180         }

```

```

177     }
178
179     struct sockaddr_in server;
180     server.sin_family = AF_INET;
181     server.sin_port = htons(80);
182     server.sin_addr.s_addr = *(unsigned*) (he->h_addr);
183
184     t = connect(s3, (struct sockaddr*) &server, sizeof(struct
185         sockaddr_in));
186     if (t == -1) {
187         perror("Connect fails");
188         exit(1);
189     }
190
191     // Inoltro la richiesta al server.
192     sprintf(request, "GET %s
193         HTTP/1.1\r\nHost:%s\r\nConnection:close\r\n\r\n", filename,
194         hostname);
195     write(s3, request, strlen(request));
196
197     // Inoltro la risposta del server al client.
198     char buffer[2001];
199     while (t = read(s3, buffer, 2000)) {
200         write(s2, buffer, t);
201     }
202     close(s3);
203     exit(0);
204
205 } else if (!strcmp(method, "CONNECT")) {
206     char* hostname = url;
207     // Riferimento: connect hostname:port
208     for (i = 0; url[i] != ':' && url[i]; i++) ;
209     url[i] = 0;
210     char* port = url + i + 1;
211     printf("hostname: %s, port: %s", hostname, port);
212
213     // Risolvo il nome.
214     struct hostent* he = gethostbyname(hostname);
215     printf("ip server: %d.%d.%d.%d\n", (unsigned char) he->h_addr[0],
216         (unsigned char) he->h_addr[1], (unsigned char) he->h_addr[2],
217         (unsigned char) he->h_addr[3]);
218
219     // Creazione client del proxy.
220     int s3 = socket(AF_INET, SOCK_STREAM, 0);
221     if (s3 == -1) {
222         perror("Socket fails");
223         exit(1);
224     }
225
226     struct sockaddr_in server;
227     server.sin_family = AF_INET;
228     server.sin_port = htons(atoi(port));
229     server.sin_addr.s_addr = *(unsigned*) (he->h_addr);
230
231     t = connect(s3, (struct sockaddr*) &server, sizeof(struct
232         sockaddr_in));

```

```
227     if (t == -1) {
228         perror("Connect fails");
229         exit(1);
230     }
231     sprintf(response, "HTTP/1.1 200 Established\r\n\r\n");
232
233     char response2[2001];
234     char request2[2001];
235     int pid = fork();
236     if (!pid) { // Child.
237         while (t = read(s2, request2, 2000)) {
238             write(s3, request2, t);
239         }
240         exit(0);
241     } else { // Parent.
242         while (t = read(s3, response2, 2000)) {
243             write(s2, response2, t);
244         }
245     }
246     kill(pid, SIGTERM);
247     close(s3);
248 }
249
250
251 return 0;
252 }
```